

TFG V6.5 - Code Line-by-Line Guide

This PDF explains the delivered Python script line by line.

The code follows the Perception - Decision - Actuation structure used in the thesis.

The explanation is intentionally direct: each row gives the line number, the exact code line, and its purpose.

Total code lines explained: 1526.

Line	Code	Explanation
1	import csv	Permite escribir los registros experimentales en archivos CSV.
2	import json	Permite guardar la configuracion de la ejecucion en JSON.
3	import time	Proporciona tiempos, esperas y duraciones del experimento.
4	import traceback	Permite imprimir errores completos durante la ejecucion.
5	from collections import deque	Estructura de cola usada para guardar el historial de areas.
6	from datetime import datetime	Permite generar marcas temporales legibles.
7	from pathlib import Path	Gestiona rutas y carpetas de forma portable.
8		Linea en blanco para separar bloques logicos del codigo.
9	import cv2	OpenCV se usa para imagen, video, ventana y dibujos.
10	from djitellopy import Tello	Clase de djitellopy usada para comunicar con el dron.
11	from ultralytics import YOLO	Clase de Ultralytics usada para cargar y ejecutar el detector.
12		Linea en blanco para separar bloques logicos del codigo.
13	# Profile	Comentario corto que identifica la seccion del codigo.
14	PROFILE_NAME = "V6_5_HEAD_ZONE_UPWARD_AVOIDANCE"	Nombre tecnico del perfil experimental que queda guardado en los logs.
15	PROFILE_NOTE = (	Nota breve que describe el comportamiento principal de esta version.
16	"Batch-labelled experimental version with upward escape for a centred person/head. "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
17	"Downward branch disabled. Operator labels: Y, F, U."	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
18	)	Cierra una estructura o llamada multilínea.
19		Linea en blanco para separar bloques logicos del codigo.
20		Linea en blanco para separar bloques logicos del codigo.
21	# Configuration	Comentario corto que identifica la seccion del codigo.
22	MODEL_PATH = "yolo11n.pt"	Ruta del modelo YOLO usado para la inferencia.
23	BASE_OUTPUT_DIR = "tello_quantitative_trials_v6_5_bidirectional_vertical_escape"	Carpeta base donde se guardan videos, imagenes y CSV de cada ejecucion.
24	WINDOW_NAME = "TFG V6.5 - Bidirectional Vertical Escape Test"	Título de la ventana de OpenCV.
25		Linea en blanco para separar bloques logicos del codigo.
26	TARGET_CLASSES = {"person", "car", "truck", "bus", "bicycle", "motorcycle"}	Clases de YOLO consideradas como obstaculos relevantes.
27	CONF_THRES = 0.35	Confianza minima aceptada para una deteccion.
28	SHOW_NON_TARGETS = False	Controla si se muestran tambien detecciones que no son obstaculos objetivo.
29		Linea en blanco para separar bloques logicos del codigo.
30		Linea en blanco para separar bloques logicos del codigo.
31	# Safety corridor	Comentario corto que identifica la seccion del codigo.
32	CENTER_X_MIN = 0.32	Limite izquierdo de la region central de avance.
33	CENTER_X_MAX = 0.68	Limite derecho de la region central de avance.
34	SAFETY_X_MIN = 0.28	Limite izquierdo del corredor de seguridad.
35	SAFETY_X_MAX = 0.72	Limite derecho del corredor de seguridad.
36	EXIT_MARGIN = 0.00	Margen adicional para considerar que el obstaculo ha salido del corredor.
37		Linea en blanco para separar bloques logicos del codigo.
38		Linea en blanco para separar bloques logicos del codigo.

Line	Code	Explanation
39	# Decision thresholds	Comentario corto que identifica la seccion del codigo.
40	AREA_WARNING = 0.550	Area normalizada que activa el estado de advertencia.
41	AREA_TRIGGER = 0.700	Area normalizada que puede activar la maniobra lateral.
42	AREA_HARD_STOP = 0.920	Area normalizada que activa una reaccion de parada o retroceso.
43		Linea en blanco para separar bloques logicos del codigo.
44		Linea en blanco para separar bloques logicos del codigo.
45	GROWTH_TRIGGER = 0.350	Crecimiento minimo del area para confirmar que el obstaculo se aproxima.
46	AREA_HISTORY_LEN = 6	Numero de muestras usadas para calcular crecimiento temporal.
47	CONFIRM_FRAMES_REQUIRED = 3	Frames consecutivos necesarios para confirmar riesgo.
48	EXIT_CORRIDOR_FRAMES_REQUIRED = 2	Frames necesarios para confirmar salida del corredor.
49	LOST_CLEAR_FRAMES_REQUIRED = 6	Frames sin deteccion necesarios para considerar despejado.
50	CLEAR_AREA_MAX = 0.025	Area maxima para considerar que una deteccion residual es pequena.
51		Linea en blanco para separar bloques logicos del codigo.
52		Linea en blanco para separar bloques logicos del codigo.
53	BYPASS_RECHECK_AREA_MIN = 0.120	Area minima para volver a revisar obstaculo durante bypass.
54		Linea en blanco para separar bloques logicos del codigo.
55		Linea en blanco para separar bloques logicos del codigo.
56	USE_TTC_TRIGGER = False	Activa o desactiva el disparo por proxy de TTC.
57	TTC_TRIGGER_SEC = 1.40	Umbral del proxy de tiempo hasta colision.
58	MIN_TTC_AREA = 0.180	Area minima para aceptar el criterio TTC.
59	MIN_AREA_GROWTH_RATE = 0.080	Tasa minima de crecimiento para aceptar el criterio TTC.
60		Linea en blanco para separar bloques logicos del codigo.
61		Linea en blanco para separar bloques logicos del codigo.
62	# Distance proxy	Comentario corto que identifica la seccion del codigo.
63	USE_DISTANCE_TRIGGER = True	Activa la estimacion de distancia basada en area calibrada.
64	REF_DISTANCE_M = 0.50	Distancia de referencia usada en la calibracion monocular.
65	AREA_REF_AT_DIST = 0.800	Area observada a la distancia de referencia.
66	TRIGGER_DISTANCE_M = 0.45	Distancia estimada que puede activar evitacion.
67	HARD_STOP_DISTANCE_M = 0.30	Distancia estimada que activa parada de emergencia.
68		Linea en blanco para separar bloques logicos del codigo.
69		Linea en blanco para separar bloques logicos del codigo.
70	# Timing limits	Comentario corto que identifica la seccion del codigo.
71	TAKEOFF_STABILIZE_SEC = 2.0	Tiempo de estabilizacion despues del despegue.
72	MIN_BATTERY_PERCENT = 20	Bateria minima permitida para iniciar pruebas.
73	MIN_APPROACH_SEC = 1.20	Tiempo minimo de avance antes de permitir evitacion normal.
74	MAX_TRIGGER_WAIT_SEC = 24.0	Tiempo maximo de espera antes de cerrar un intento sin disparo.
75	BRAKE_BEFORE_AVOID_SEC = 0.40	Pausa antes de ejecutar la maniobra de evitacion.
76	MIN_LATERAL_SEC = 0.35	Duracion minima del movimiento lateral.

Line	Code	Explanation
77	MAX_LATERAL_SEC = 6.00	Duracion maxima del movimiento lateral.
78		Linea en blanco para separar bloques logicos del codigo.
79		Linea en blanco para separar bloques logicos del codigo.
80	BYPASS_FORWARD_SEC = 8.00	Duracion de bypass frontal cuando no se usa estimacion por distancia.
81		Linea en blanco para separar bloques logicos del codigo.
82		Linea en blanco para separar bloques logicos del codigo.
83	RECENTER_SEC = 1.50	Tiempo base para recentrar cuando no se usa esfuerzo lateral.
84		Linea en blanco para separar bloques logicos del codigo.
85		Linea en blanco para separar bloques logicos del codigo.
86	FINAL_FORWARD_SEC = 3.00	Tiempo de avance final despues de recentrar.
87		Linea en blanco para separar bloques logicos del codigo.
88		Linea en blanco para separar bloques logicos del codigo.
89	MIN_BYPASS_BEFORE_RECHECK_SEC = 1.20	Tiempo minimo antes de volver a revisar durante bypass.
90		Linea en blanco para separar bloques logicos del codigo.
91		Linea en blanco para separar bloques logicos del codigo.
92	# Bypass and recenter	Comentario corto que identifica la seccion del codigo.
93	USE_DISTANCE_BASED_BYPASS = True	Activa el bypass calculado por distancia estimada.
94	BYPASS_TARGET_DISTANCE_M = 2.50	Distancia objetivo del tramo de bypass.
95	FB_MPS_AT_RC20 = 0.30	Velocidad aproximada para convertir comando RC en m/s.
96	MIN_BYPASS_FORWARD_SEC = 3.00	Duracion minima del bypass frontal.
97	MAX_BYPASS_FORWARD_SEC = 12.00	Duracion maxima del bypass frontal.
98		Linea en blanco para separar bloques logicos del codigo.
99		Linea en blanco para separar bloques logicos del codigo.
100	USE_EFFORT_BASED_RECENTER = True	Activa el recentrado calculado por esfuerzo lateral acumulado.
101	RECENTER_COMPENSATION = 1.10	Factor de compensacion aplicado al recentrado.
102	MIN_RECENTER_SEC = 1.50	Duracion minima de recentrado.
103	MAX_RECENTER_SEC = 5.00	Duracion maxima de recentrado.
104		Linea en blanco para separar bloques logicos del codigo.
105		Linea en blanco para separar bloques logicos del codigo.
106	# Bypass mode	Comentario corto que identifica la seccion del codigo.
107	BYPASS_MODE = "straight"	Modo de bypass seleccionado.
108	BYPASS_DIAGONAL_LR = 0	Componente lateral del bypass diagonal.
109	BYPASS_DIAGONAL_SEC = 0.0	Duracion del componente diagonal.
110		Linea en blanco para separar bloques logicos del codigo.
111		Linea en blanco para separar bloques logicos del codigo.
112	# Vertical avoidance	Comentario corto que identifica la seccion del codigo.
113	ENABLE_UPWARD_AVOIDANCE = True	Activa la rama de evitacion vertical ascendente.
114	UP_ONLY_FOR_CLASSES = {"person"}	Clases para las que se permite la evitacion ascendente.

Line	Code	Explanation
115		Linea en blanco para separar bloques logicos del codigo.
116		Linea en blanco para separar bloques logicos del codigo.
117	UP_AREA_TRIGGER = 0.400	Area minima para considerar subida.
118	UP_DISTANCE_TRIGGER_M = 0.75	Distancia estimada para considerar subida.
119	UP_MAX_AREA = 0.900	Area maxima para permitir subida sin estar demasiado cerca.
120		Linea en blanco para separar bloques logicos del codigo.
121		Linea en blanco para separar bloques logicos del codigo.
122	UP_CENTER_MAX_OFFSET = 0.22	Desplazamiento horizontal maximo para considerar subida.
123	UP_MIN_TOP_CLEARANCE = 0.18	Parametro auxiliar de margen superior.
124	UP_MAX_BBOX_HEIGHT_NORM = 0.88	Altura maxima normalizada del bbox para permitir subida.
125	UP_CLEAR_TOP_Y_NORM = 0.50	Umbral visual para considerar espacio superior.
126	UP_LATERAL_EDGE_BIAS = 0.00	Sesgo para evitar bloquear subida por jitter lateral.
127		Linea en blanco para separar bloques logicos del codigo.
128		Linea en blanco para separar bloques logicos del codigo.
129	UP_HEAD_Y1_MAX = 0.25	Umbral superior del bbox para detectar cabeza o zona alta.
130	UP_HEAD_CENTER_Y_MAX = 0.55	Umbral del centro vertical para detectar zona alta.
131		Linea en blanco para separar bloques logicos del codigo.
132		Linea en blanco para separar bloques logicos del codigo.
133	UP_MAX_HEIGHT_CM = 170	Altura maxima permitida para ejecutar subida.
134		Linea en blanco para separar bloques logicos del codigo.
135		Linea en blanco para separar bloques logicos del codigo.
136	RC_UP_AVOID = 22	Comando RC vertical usado en la subida.
137	UP_AVOID_MIN_SEC = 0.70	Duracion minima de subida.
138	UP_AVOID_TARGET_SEC = 1.15	Duracion objetivo de subida.
139	UP_AVOID_MAX_SEC = 1.80	Duracion maxima de subida.
140		Linea en blanco para separar bloques logicos del codigo.
141		Linea en blanco para separar bloques logicos del codigo.
142	USE_DOWN_RECENTER_AFTER_UP = False	Controla si baja automaticamente despues de subir.
143	DOWN_RECENTER_SEC = 0.60	Duracion de bajada tras subida si se activa.
144	RC_DOWN_RECENTER = 12	Comando RC de bajada tras subida.
145		Linea en blanco para separar bloques logicos del codigo.
146		Linea en blanco para separar bloques logicos del codigo.
147	# Downward test branch	Comentario corto que identifica la seccion del codigo.
148	ENABLE_DOWNWARD_AVOIDANCE = False	Activa la rama experimental descendente.
149	DOWN_ONLY_FOR_CLASSES = {"person"}	Clases para las que se permite bajada.
150		Linea en blanco para separar bloques logicos del codigo.
151		Linea en blanco para separar bloques logicos del codigo.
152	DOWN_AREA_TRIGGER = 0.520	Area minima para considerar bajada.

Line	Code	Explanation
153	DOWN_DISTANCE_TRIGGER_M = 0.60	Distancia estimada para considerar bajada.
154	DOWN_MAX_AREA = 0.850	Area maxima para permitir bajada.
155	DOWN_CENTER_MAX_OFFSET = 0.16	Desplazamiento horizontal maximo para bajar.
156	DOWN_MAX_CENTER_Y = 0.48	Centro vertical maximo para considerar obstaculo alto.
157	DOWN_MIN_BOTTOM_CLEARANCE = 0.18	Espacio inferior minimo en imagen para bajar.
158	DOWN_MAX_BBOX_HEIGHT_NORM = 0.82	Altura maxima del bbox para bajar.
159	DOWN_CLEAR_BOTTOM_Y_NORM = 0.50	Umbral visual de espacio inferior.
160	DOWN_LATERAL_EDGE_BIAS = 0.09	Sesgo lateral de la rama descendente.
161		Linea en blanco para separar bloques logicos del codigo.
162		Linea en blanco para separar bloques logicos del codigo.
163	DOWN_MIN_HEIGHT_CM = 90	Altura minima para permitir bajada.
164		Linea en blanco para separar bloques logicos del codigo.
165		Linea en blanco para separar bloques logicos del codigo.
166	RC_DOWN_AVOID = 18	Comando RC vertical descendente.
167	DOWN_AVOID_MIN_SEC = 0.50	Duracion minima de bajada.
168	DOWN_AVOID_TARGET_SEC = 0.85	Duracion objetivo de bajada.
169	DOWN_AVOID_MAX_SEC = 1.30	Duracion maxima de bajada.
170		Linea en blanco para separar bloques logicos del codigo.
171		Linea en blanco para separar bloques logicos del codigo.
172	USE_UP_RECENTER_AFTER_DOWN = False	Controla si sube despues de una bajada.
173	UP_RECENTER_AFTER_DOWN_SEC = 0.60	Duracion de subida tras bajada.
174	RC_UP_RECENTER_AFTER_DOWN = 12	Comando RC de subida tras bajada.
175		Linea en blanco para separar bloques logicos del codigo.
176		Linea en blanco para separar bloques logicos del codigo.
177	# RC command values	Comentario corto que identifica la seccion del codigo.
178	RC_MANUAL_SPEED = 28	Velocidad manual para desplazamientos por teclado.
179	RC_YAW_SPEED = 35	Velocidad de yaw manual.
180	RC_UPDOWN_SPEED = 28	Velocidad vertical manual.
181	RC_APPROACH_FORWARD = 14	Comando de avance durante aproximacion.
182	RC_BYPASS_FORWARD = 22	Comando de avance durante bypass.
183	RC_FINAL_FORWARD = 18	Comando de avance final.
184	RC_LATERAL_BASE = 24	Comando lateral base.
185	RC_LATERAL_STRONG = 30	Comando lateral reforzado.
186	RC_LATERAL_MAX = 36	Comando lateral maximo.
187	RC_RECENTER = 16	Comando lateral usado para recentrar.
188	CONTROL_ALPHA = 0.65	Factor de suavizado de comandos RC.
189	EMERGENCY_BACK_SPEED = -8	Comando de retroceso ante riesgo extremo.
190		Linea en blanco para separar bloques logicos del codigo.

Line	Code	Explanation
191	# Output logging	Comentario corto que identifica la seccion del codigo.
192	SAVE_ANNOTATED_VIDEO = True	Activa guardado del video anotado.
193	SAVE_RAW_VIDEO = True	Activa guardado del video bruto.
194	SAVE_PERIODIC_FRAMES = True	Activa guardado periodico de frames.
195	PERIODIC_FRAME_INTERVAL_SEC = 0.7	Intervalo entre frames guardados.
196	VIDEO_FPS = 20	FPS usado para escribir los videos.
197	TELEMETRY_LOG_INTERVAL_SEC = 0.3	Intervalo de registro de telemetria.
198		Linea en blanco para separar bloques logicos del codigo.
199	# Keyboard codes	Comentario corto que identifica la seccion del codigo.
200	ARROW_LEFT = 2424832	Codigo de teclado para flecha izquierda.
201	ARROW_UP = 2490368	Codigo de teclado para flecha arriba.
202	ARROW_RIGHT = 2555904	Codigo de teclado para flecha derecha.
203	ARROW_DOWN = 2621440	Codigo de teclado para flecha abajo.
204		Linea en blanco para separar bloques logicos del codigo.
205		Linea en blanco para separar bloques logicos del codigo.
206	# Utility functions	Comentario corto que identifica la seccion del codigo.
207	def now_iso() -> str:	Define la funcion now_iso.
208	return datetime.now().isoformat(timespec="milliseconds")	Devuelve el resultado calculado por la funcion.
209		Linea en blanco para separar bloques logicos del codigo.
210		Linea en blanco para separar bloques logicos del codigo.
211	def make_run_dir(base_dir: str) -> Path:	Define la funcion make_run_dir.
212	ts = datetime.now().strftime("%Y%m%d_%H%M%S")	Calcula o inicializa la variable ts.
213	run_dir = Path(base_dir) / f"run_{ts}"	Calcula o inicializa la variable run_dir.
214	(run_dir / "frames_raw").mkdir(parents=True, exist_ok=True)	Crea la carpeta necesaria para guardar resultados.
215	(run_dir / "frames_annotated").mkdir(parents=True, exist_ok=True)	Crea la carpeta necesaria para guardar resultados.
216	return run_dir	Devuelve el resultado calculado por la funcion.
217		Linea en blanco para separar bloques logicos del codigo.
218		Linea en blanco para separar bloques logicos del codigo.
219	def safe_call(fn, default=-1):	Define la funcion safe_call.
220	try:	Inicia un bloque protegido frente a errores de ejecucion.
221	return fn()	Devuelve el resultado calculado por la funcion.
222	except Exception:	Gestiona una excepcion producida dentro del bloque try.
223	return default	Devuelve el resultado calculado por la funcion.
224		Linea en blanco para separar bloques logicos del codigo.
225		Linea en blanco para separar bloques logicos del codigo.
226	def open_csv_writer(path: Path, header):	Define la funcion open_csv_writer.
227	f = open(path, "w", newline="", encoding="utf-8")	Calcula o inicializa la variable f.
228	writer = csv.writer(f)	Calcula o inicializa la variable writer.

Line	Code	Explanation
229	<code>writer.writerow(header)</code>	Escribe una fila en un archivo CSV de registro.
230	<code>f.flush()</code>	Fuerza el guardado inmediato de los datos pendientes.
231	<code>return f, writer</code>	Devuelve el resultado calculado por la funcion.
232		Linea en blanco para separar bloques logicos del codigo.
233		Linea en blanco para separar bloques logicos del codigo.
234	<code>def safe_stop_rc(tello: Tello):</code>	Define la funcion <code>safe_stop_rc</code> .
235	<code>try:</code>	Inicia un bloque protegido frente a errores de ejecucion.
236	<code>tello.send_rc_control(0, 0, 0, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
237	<code>except Exception:</code>	Gestiona una excepcion producida dentro del bloque try.
238	<code>pass</code>	Ejecuta una instruccion necesaria para el flujo del programa.
239		Linea en blanco para separar bloques logicos del codigo.
240		Linea en blanco para separar bloques logicos del codigo.
241	<code>def smooth(prev, new, alpha=CONTROL_ALPHA):</code>	Define la funcion <code>smooth</code> .
242	<code>return int(alpha * prev + (1.0 - alpha) * new)</code>	Devuelve el resultado calculado por la funcion.
243		Linea en blanco para separar bloques logicos del codigo.
244		Linea en blanco para separar bloques logicos del codigo.
245	<code>def bbox_intersects_safety_corridor(det) -> bool:</code>	Define la funcion <code>bbox_intersects_safety_corridor</code> .
246	<code>if det is None:</code>	Evalua una condicion antes de ejecutar el bloque siguiente.
247	<code>return False</code>	Devuelve el resultado calculado por la funcion.
248	<code>return (det["x2_norm"] >= SAFETY_X_MIN) and (det["x1_norm"] <= SAFETY_X_MAX)</code>	Devuelve el resultado calculado por la funcion.
249		Linea en blanco para separar bloques logicos del codigo.
250		Linea en blanco para separar bloques logicos del codigo.
251	<code>def bbox_outside_safety_corridor(det) -> bool:</code>	Define la funcion <code>bbox_outside_safety_corridor</code> .
252		Linea en blanco para separar bloques logicos del codigo.
253		Linea en blanco para separar bloques logicos del codigo.
254	<code>if det is None:</code>	Evalua una condicion antes de ejecutar el bloque siguiente.
255	<code>return False</code>	Devuelve el resultado calculado por la funcion.
256	<code>left_clear = det["x2_norm"] < (SAFETY_X_MIN - EXIT_MARGIN)</code>	Calcula o inicializa la variable <code>left_clear</code> .
257	<code>right_clear = det["x1_norm"] > (SAFETY_X_MAX + EXIT_MARGIN)</code>	Calcula o inicializa la variable <code>right_clear</code> .
258	<code>return left_clear or right_clear</code>	Devuelve el resultado calculado por la funcion.
259		Linea en blanco para separar bloques logicos del codigo.
260		Linea en blanco para separar bloques logicos del codigo.
261	<code>def estimate_distance_from_area(area_norm: float):</code>	Define la funcion <code>estimate_distance_from_area</code> .
262	<code>if area_norm <= 1e-6 or AREA_REF_AT_DIST <= 1e-6:</code>	Evalua una condicion antes de ejecutar el bloque siguiente.
263	<code>return None</code>	Devuelve el resultado calculado por la funcion.
264	<code>return REF_DISTANCE_M * (AREA_REF_AT_DIST / area_norm) ** 0.5</code>	Devuelve el resultado calculado por la funcion.
265		Linea en blanco para separar bloques logicos del codigo.
266		Linea en blanco para separar bloques logicos del codigo.

Line	Code	Explanation
267	def estimate_forward_mps(rc_forward: int) -> float:	Define la funcion estimate_forward_mps.
268		Linea en blanco para separar bloques logicos del codigo.
269		Linea en blanco para separar bloques logicos del codigo.
270	return FB_MPS_AT_RC20 * (abs(rc_forward) / 20.0)	Devuelve el resultado calculado por la funcion.
271		Linea en blanco para separar bloques logicos del codigo.
272		Linea en blanco para separar bloques logicos del codigo.
273	def choose_avoid_direction(primary) -> int:	Define la funcion choose_avoid_direction.
274		Linea en blanco para separar bloques logicos del codigo.
275		Linea en blanco para separar bloques logicos del codigo.
276	if primary is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
277	return +1	Devuelve el resultado calculado por la funcion.
278	return +1 if primary["center_x_norm"] < 0.50 else -1	Devuelve el resultado calculado por la funcion.
279		Linea en blanco para separar bloques logicos del codigo.
280		Linea en blanco para separar bloques logicos del codigo.
281	def compute_area_growth_rate(area_history):	Define la funcion compute_area_growth_rate.
282		Linea en blanco para separar bloques logicos del codigo.
283	if len(area_history) < 2:	Evalua una condicion antes de ejecutar el bloque siguiente.
284	return 0.0	Devuelve el resultado calculado por la funcion.
285	t0, a0 = area_history[0]	Ejecuta una instruccion necesaria para el flujo del programa.
286	t1, a1 = area_history[-1]	Ejecuta una instruccion necesaria para el flujo del programa.
287	dt = max(1e-3, t1 - t0)	Calcula un valor maximo y lo asigna a dt.
288	return (a1 - a0) / dt	Devuelve el resultado calculado por la funcion.
289		Linea en blanco para separar bloques logicos del codigo.
290		Linea en blanco para separar bloques logicos del codigo.
291	def compute_ttc_proxy(area_norm, area_growth_rate):	Define la funcion compute_ttc_proxy.
292		Linea en blanco para separar bloques logicos del codigo.
293	if area_growth_rate <= 1e-6 or area_norm <= 1e-6:	Evalua una condicion antes de ejecutar el bloque siguiente.
294	return None	Devuelve el resultado calculado por la funcion.
295	return area_norm / area_growth_rate	Devuelve el resultado calculado por la funcion.
296		Linea en blanco para separar bloques logicos del codigo.
297		Linea en blanco para separar bloques logicos del codigo.
298	def bbox_height_norm(det) -> float:	Define la funcion bbox_height_norm.
299	if det is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
300	return 0.0	Devuelve el resultado calculado por la funcion.
301	return max(0.0, det.get("y2_norm", 0.0) - det.get("y1_norm", 0.0))	Devuelve el resultado calculado por la funcion.
302		Linea en blanco para separar bloques logicos del codigo.
303		Linea en blanco para separar bloques logicos del codigo.
304	def obstacle_below_vertical_clearance(det) -> bool:	Define la funcion obstacle_below_vertical_clearance.

Line	Code	Explanation
305		Linea en blanco para separar bloques logicos del codigo.
306		Linea en blanco para separar bloques logicos del codigo.
307	if det is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
308	return False	Devuelve el resultado calculado por la funcion.
309	return det.get("y1_norm", 0.0) >= UP_CLEAR_TOP_Y_NORM	Devuelve el resultado calculado por la funcion.
310		Linea en blanco para separar bloques logicos del codigo.
311		Linea en blanco para separar bloques logicos del codigo.
312	def obstacle_above_vertical_clearance(det) -> bool:	Define la funcion obstacle_above_vertical_clearance.
313		Linea en blanco para separar bloques logicos del codigo.
314		Linea en blanco para separar bloques logicos del codigo.
315	if det is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
316	return False	Devuelve el resultado calculado por la funcion.
317	return det.get("y2_norm", 1.0) <= DOWN_CLEAR_BOTTOM_Y_NORM	Devuelve el resultado calculado por la funcion.
318		Linea en blanco para separar bloques logicos del codigo.
319		Linea en blanco para separar bloques logicos del codigo.
320	def choose_avoidance_strategy(primary, current_area, distance_est, height_cm):	Define la funcion choose_avoidance_strategy.
321		Linea en blanco para separar bloques logicos del codigo.
322		Linea en blanco para separar bloques logicos del codigo.
323	lateral_sign = choose_avoid_direction(primary)	Calcula o inicializa la variable lateral_sign.
324	lateral_text = "right" if lateral_sign == +1 else "left"	Calcula o inicializa la variable lateral_text.
325		Linea en blanco para separar bloques logicos del codigo.
326	if primary is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
327	return "lateral", lateral_sign, lateral_text, "no_primary_detection"	Devuelve el resultado calculado por la funcion.
328		Linea en blanco para separar bloques logicos del codigo.
329	if not bbox_intersects_safety_corridor(primary):	Evalua una condicion antes de ejecutar el bloque siguiente.
330	return "lateral", lateral_sign, lateral_text, "outside_safety_corridor"	Devuelve el resultado calculado por la funcion.
331		Linea en blanco para separar bloques logicos del codigo.
332	cx = primary["center_x_norm"]	Calcula o inicializa la variable cx.
333	cy = primary.get("center_y_norm", 0.5)	Calcula o inicializa la variable cy.
334	y1n = primary.get("y1_norm", 0.0)	Calcula o inicializa la variable y1n.
335	y2n = primary.get("y2_norm", 1.0)	Calcula o inicializa la variable y2n.
336	h_norm = bbox_height_norm(primary)	Calcula o inicializa la variable h_norm.
337		Linea en blanco para separar bloques logicos del codigo.
338	lateral_easy = (cx < CENTER_X_MIN + UP_LATERAL_EDGE_BIAS) or (cx > CENTER_X_MAX - UP_LATERAL_EDGE_BIAS)	Calcula o inicializa la variable lateral_easy.
339		Linea en blanco para separar bloques logicos del codigo.
340		Linea en blanco para separar bloques logicos del codigo.
341	if ENABLE_DOWNWARD_AVOIDANCE and primary["class_name"] in DOWN_ONLY_FOR_CLASSES:	Evalua una condicion antes de ejecutar el bloque siguiente.
342	down_close_enough = (	Calcula o inicializa la variable down_close_enough.

Line	Code	Explanation
343	current_area >= DOWN_AREA_TRIGGER or	Ejecuta una instruccion necesaria para el flujo del programa.
344	(distance_est is not None and distance_est <= DOWN_DISTANCE_TRIGGER_M)	Ejecuta una instruccion necesaria para el flujo del programa.
345	)	Cierra una estructura o llamada multilinea.
346	down_not_too_close = current_area <= DOWN_MAX_AREA	Calcula o inicializa la variable down_not_too_close.
347	down_centred = abs(cx - 0.50) <= DOWN_CENTER_MAX_OFFSET	Calcula o inicializa la variable down_centred.
348	enough_space_below = (1.0 - y2n) >= DOWN_MIN_BOTTOM_CLEARANCE	Calcula o inicializa la variable enough_space_below.
349	obstacle_high = cy <= DOWN_MAX_CENTER_Y	Calcula o inicializa la variable obstacle_high.
350	down_not_full_height = h_norm <= DOWN_MAX_BBOX_HEIGHT_NORM	Calcula o inicializa la variable down_not_full_height.
351		Linea en blanco para separar bloques logicos del codigo.
352	if height_cm is None or height_cm < 0:	Evalua una condicion antes de ejecutar el bloque siguiente.
353	down_altitude_ok = False	Inicializa la bandera booleana down_altitude_ok.
354	down_altitude_note = "height_unknown_down_disabled"	Calcula o inicializa la variable down_altitude_note.
355	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
356	down_altitude_ok = height_cm >= DOWN_MIN_HEIGHT_CM	Calcula o inicializa la variable down_altitude_ok.
357	down_altitude_note = f"height={height_cm}cm"	Construye una cadena formateada y la asigna a down_altitude_note.
358		Linea en blanco para separar bloques logicos del codigo.
359	if (down_close_enough and down_not_too_close and down_centred and	Evalua una condicion antes de ejecutar el bloque siguiente.
360	enough_space_below and obstacle_high and down_not_full_height and	Ejecuta una instruccion necesaria para el flujo del programa.
361	down_altitude_ok and not lateral_easy):	Ejecuta una instruccion necesaria para el flujo del programa.
362	reason = (	Calcula o inicializa la variable reason.
363	f"down_selected A={current_area:.3f}, cx={cx:.2f}, cy={cy:.2f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
364	f"y2={y2n:.2f}, h={h_norm:.2f}, {down_altitude_note}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
365	)	Cierra una estructura o llamada multilinea.
366	return "down", 0, "down", reason	Devuelve el resultado calculado por la funcion.
367		Linea en blanco para separar bloques logicos del codigo.
368		Linea en blanco para separar bloques logicos del codigo.
369	if ENABLE_UPWARD_AVOIDANCE and primary["class_name"] in UP_ONLY_FOR_CLASSES:	Evalua una condicion antes de ejecutar el bloque siguiente.
370	up_close_enough = (	Calcula o inicializa la variable up_close_enough.
371	current_area >= UP_AREA_TRIGGER or	Ejecuta una instruccion necesaria para el flujo del programa.
372	(distance_est is not None and distance_est <= UP_DISTANCE_TRIGGER_M)	Ejecuta una instruccion necesaria para el flujo del programa.
373	)	Cierra una estructura o llamada multilinea.
374	up_not_too_close = current_area <= UP_MAX_AREA	Calcula o inicializa la variable up_not_too_close.
375		Linea en blanco para separar bloques logicos del codigo.
376		Linea en blanco para separar bloques logicos del codigo.
377	up_centred = abs(cx - 0.50) <= UP_CENTER_MAX_OFFSET	Calcula o inicializa la variable up_centred.
378		Linea en blanco para separar bloques logicos del codigo.
379		Linea en blanco para separar bloques logicos del codigo.
380	head_or_upper_zone = (	Calcula o inicializa la variable head_or_upper_zone.

Line	Code	Explanation
381	y1n <= UP_HEAD_Y1_MAX or	Ejecuta una instruccion necesaria para el flujo del programa.
382	cy <= UP_HEAD_CENTER_Y_MAX	Ejecuta una instruccion necesaria para el flujo del programa.
383	)	Cierra una estructura o llamada multilinea.
384		Linea en blanco para separar bloques logicos del codigo.
385	up_not_full_height = h_norm <= UP_MAX_BBOX_HEIGHT_NORM	Calcula o inicializa la variable up_not_full_height.
386		Linea en blanco para separar bloques logicos del codigo.
387	if height_cm is None or height_cm < 0:	Evalua una condicion antes de ejecutar el bloque siguiente.
388	up_altitude_ok = True	Inicializa la bandera booleana up_altitude_ok.
389	up_altitude_note = "height_unknown"	Calcula o inicializa la variable up_altitude_note.
390	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
391	up_altitude_ok = height_cm <= UP_MAX_HEIGHT_CM	Calcula o inicializa la variable up_altitude_ok.
392	up_altitude_note = f"height={height_cm}cm"	Construye una cadena formateada y la asigna a up_altitude_note.
393		Linea en blanco para separar bloques logicos del codigo.
394	if (	Evalua una condicion antes de ejecutar el bloque siguiente.
395	up_close_enough and	Ejecuta una instruccion necesaria para el flujo del programa.
396	up_not_too_close and	Ejecuta una instruccion necesaria para el flujo del programa.
397	up_centred and	Ejecuta una instruccion necesaria para el flujo del programa.
398	head_or_upper_zone and	Ejecuta una instruccion necesaria para el flujo del programa.
399	up_not_full_height and	Ejecuta una instruccion necesaria para el flujo del programa.
400	up_altitude_ok	Ejecuta una instruccion necesaria para el flujo del programa.
401	):	Cierra una expresion multilinea y continua el flujo del codigo.
402	reason = (	Calcula o inicializa la variable reason.
403	f"up_selected_HEAD_ZONE A={current_area:.3f}, cx={cx:.2f}, cy={cy:.2f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
404	f"y1={y1n:.2f}, y2={y2n:.2f}, h={h_norm:.2f}, {up_altitude_note}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
405	)	Cierra una estructura o llamada multilinea.
406	return "up", 0, "up", reason	Devuelve el resultado calculado por la funcion.
407		Linea en blanco para separar bloques logicos del codigo.
408	reason = (	Calcula o inicializa la variable reason.
409	f"lateral_selected A={current_area:.3f}, cx={cx:.2f}, cy={cy:.2f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
410	f"y1={y1n:.2f}, y2={y2n:.2f}, h={h_norm:.2f}, lateral_easy={lateral_easy}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
411	)	Cierra una estructura o llamada multilinea.
412	return "lateral", lateral_sign, lateral_text, reason	Devuelve el resultado calculado por la funcion.
413		Linea en blanco para separar bloques logicos del codigo.
414	def draw_overlay(	Define la funcion draw_overlay.
415	frame,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
416	flying,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
417	state,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
418	scenario_name,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.

Line	Code	Explanation
419	attempt_active,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
420	primary,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
421	distance_est,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
422	ttc_proxy,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
423	danger,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
424	warning,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
425	exit_counter,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
426	lost_counter,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
427	lateral_effort,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
428	recenter_target_sec,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
429	bypass_distance_est_m,	Elemento dentro de una lista, tupla, diccionario o llamada multilínea.
430	avoidance_mode="none",	Calcula o inicializa la variable avoidance_mode.
431	vertical_effort=0.0,	Calcula o inicializa la variable vertical_effort.
432	up_duration=0.0,	Calcula o inicializa la variable up_duration.
433	):	Cierra una expresion multilínea y continua el flujo del codigo.
434	lines = [	Calcula o inicializa la variable lines.
435	"T Takeoff L Land Q Quit",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
436	"Arrows move W/S up-down A/D yaw",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
437	"1/2/3/4/5 scenario Enter start attempt X abort",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
438	f"Profile: {PROFILE_NAME}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
439	f"State: {state}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
440	f"Scenario: {scenario_name}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
441	f"Attempt: {'ON' if attempt_active else 'OFF'}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
442	f"Warning: {warning} Danger: {danger} Exit frames: {exit_counter} Lost frames: {lost_counter}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
443	f"Lat effort: {lateral_effort:.1f} Recenter target: {recenter_target_sec:.2f}s",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
444	f"Bypass est: {bypass_distance_est_m:.2f}m / target {BYPASS_TARGET_DISTANCE_M:.2f}m",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
445	f"Avoid mode: {avoidance_mode} Vertical effort: {vertical_effort:.1f} Vertical sec: {up_duration:.2f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
446	"After attempt: Y success F failure U unclear",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
447	]	Cierra una estructura o llamada multilínea.
448	if primary is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
449	dist_txt = f" d~{distance_est:.2f}m" if distance_est is not None else " d-NA"	Construye una cadena formateada y la asigna a dist_txt.
450	ttc_txt = f" ttc~{ttc_proxy:.2f}s" if ttc_proxy is not None else " ttc-NA"	Construye una cadena formateada y la asigna a ttc_txt.
451	lines.append(	Anade un dato a una lista o cola.
452	f"Primary: {primary['class_name']} A={primary['bbox_area_norm']:.3f} "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
453	f"cx={primary['center_x_norm']:.2f} cy={primary['center_y_norm']:.2f} "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
454	f"x=[{primary['x1_norm']:.2f},{primary['x2_norm']:.2f}] "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
455	f"y=[{primary.get('y1_norm', 0.0):.2f},{primary.get('y2_norm', 0.0):.2f}]"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
456	+ dist_txt + ttc_txt	Ejecuta una instruccion necesaria para el flujo del programa.

Line	Code	Explanation
457	)	Cierra una estructura o llamada multilinea.
458	if not flying:	Evalua una condicion antes de ejecutar el bloque siguiente.
459	lines.append("NEXT: Press T to take off")	Anade un dato a una lista o cola.
460		Linea en blanco para separar bloques logicos del codigo.
461	y = 24	Calcula o inicializa la variable y.
462	for txt in lines:	Inicia un bucle sobre una secuencia de elementos.
463	cv2.putText(frame, txt, (12, y), cv2.FONT_HERSHEY_SIMPLEX, 0.48, (255, 255, 255), 2)	Escribe texto informativo sobre el frame.
464	y += 21	Ejecuta una instruccion necesaria para el flujo del programa.
465		Linea en blanco para separar bloques logicos del codigo.
466		Linea en blanco para separar bloques logicos del codigo.
467	# Main program	Comentario corto que identifica la seccion del codigo.
468	def main():	Define la funcion main.
469	model = YOLO(MODEL_PATH)	Carga el modelo YOLO seleccionado.
470	run_dir = make_run_dir(BASE_OUTPUT_DIR)	Calcula o inicializa la variable run_dir.
471		Linea en blanco para separar bloques logicos del codigo.
472	config = {	Calcula o inicializa la variable config.
473	"profile_name": PROFILE_NAME,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
474	"profile_note": PROFILE_NOTE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
475	"model_path": MODEL_PATH,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
476	"conf_thres": CONF_THRES,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
477	"control_style": "CLEARANCE_AROUND_OBSTACLE_RC_FSM_PROFILED",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
478	"architecture": "Perception-Decision-Actuation",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
479	"center_x_min": CENTER_X_MIN,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
480	"center_x_max": CENTER_X_MAX,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
481	"safety_x_min": SAFETY_X_MIN,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
482	"safety_x_max": SAFETY_X_MAX,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
483	"exit_margin": EXIT_MARGIN,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
484	"area_warning": AREA_WARNING,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
485	"area_trigger": AREA_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
486	"area_hard_stop": AREA_HARD_STOP,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
487	"growth_trigger": GROWTH_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
488	"confirm_frames_required": CONFIRM_FRAMES_REQUIRED,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
489	"exit_corridor_frames_required": EXIT_CORRIDOR_FRAMES_REQUIRED,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
490	"lost_clear_frames_required": LOST_CLEAR_FRAMES_REQUIRED,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
491	"clear_area_max": CLEAR_AREA_MAX,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
492	"use_ttc_trigger": USE_TTC_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
493	"ttc_trigger_sec": TTC_TRIGGER_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
494	"min_ttc_area": MIN_TTC_AREA,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.

Line	Code	Explanation
495	"min_area_growth_rate": MIN_AREA_GROWTH_RATE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
496	"use_distance_trigger": USE_DISTANCE_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
497	"ref_distance_m": REF_DISTANCE_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
498	"area_ref_at_dist": AREA_REF_AT_DIST,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
499	"trigger_distance_m": TRIGGER_DISTANCE_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
500	"hard_stop_distance_m": HARD_STOP_DISTANCE_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
501	"min_approach_sec": MIN_APPROACH_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
502	"bypass_forward_sec_fallback": BYPASS_FORWARD_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
503	"use_distance_based_bypass": USE_DISTANCE_BASED_BYPASS,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
504	"bypass_target_distance_m": BYPASS_TARGET_DISTANCE_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
505	"fb_mps_at_rc20": FB_MPS_AT_RC20,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
506	"use_effort_based_recenter": USE_EFFORT_BASED_RECENTER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
507	"recenter_compensation": RECENTER_COMPENSATION,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
508	"min_recenter_sec": MIN_RECENTER_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
509	"max_recenter_sec": MAX_RECENTER_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
510	"bypass_recheck_area_min": BYPASS_RECHECK_AREA_MIN,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
511	"bypass_mode": BYPASS_MODE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
512	"enable_upward_avoidance": ENABLE_UPWARD_AVOIDANCE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
513	"up_area_trigger": UP_AREA_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
514	"up_distance_trigger_m": UP_DISTANCE_TRIGGER_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
515	"up_min_top_clearance": UP_MIN_TOP_CLEARANCE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
516	"up_clear_top_y_norm": UP_CLEAR_TOP_Y_NORM,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
517	"up_center_max_offset": UP_CENTER_MAX_OFFSET,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
518	"up_head_y1_max": UP_HEAD_Y1_MAX,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
519	"up_head_center_y_max": UP_HEAD_CENTER_Y_MAX,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
520	"up_lateral_edge_bias": UP_LATERAL_EDGE_BIAS,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
521	"up_max_height_cm": UP_MAX_HEIGHT_CM,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
522	"rc_up_avoid": RC_UP_AVOID,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
523	"up_avoid_target_sec": UP_AVOID_TARGET_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
524	"use_down_recenter_after_up": USE_DOWN_RECENTER_AFTER_UP,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
525	"enable_downward_avoidance": ENABLE_DOWNWARD_AVOIDANCE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
526	"down_area_trigger": DOWN_AREA_TRIGGER,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
527	"down_distance_trigger_m": DOWN_DISTANCE_TRIGGER_M,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
528	"down_min_bottom_clearance": DOWN_MIN_BOTTOM_CLEARANCE,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
529	"down_clear_bottom_y_norm": DOWN_CLEAR_BOTTOM_Y_NORM,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
530	"down_center_max_offset": DOWN_CENTER_MAX_OFFSET,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
531	"down_max_center_y": DOWN_MAX_CENTER_Y,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
532	"down_min_height_cm": DOWN_MIN_HEIGHT_CM,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.

Line	Code	Explanation
533	"rc_down_avoid": RC_DOWN_AVOID,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
534	"down_avoid_target_sec": DOWN_AVOID_TARGET_SEC,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
535	"use_up_recenter_after_down": USE_UP_RECENTER_AFTER_DOWN,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
536	"created_at": now_iso(),	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
537	}	Cierra una estructura o llamada multilinea.
538	with open(run_dir / "run_config.json", "w", encoding="utf-8") as f:	Abre un recurso usando un gestor de contexto seguro.
539	json.dump(config, f, indent=2)	Guarda el diccionario de configuracion en un archivo JSON.
540		Linea en blanco para separar bloques logicos del codigo.
541	events_f, events_writer = open_csv_writer(	Ejecuta una instruccion necesaria para el flujo del programa.
542	run_dir / "events.csv",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
543	["timestamp", "frame_idx", "event", "details"]	Ejecuta una instruccion necesaria para el flujo del programa.
544	)	Cierra una estructura o llamada multilinea.
545	attempts_f, attempts_writer = open_csv_writer(	Ejecuta una instruccion necesaria para el flujo del programa.
546	run_dir / "attempts.csv",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
547	[	Ejecuta una instruccion necesaria para el flujo del programa.
548	"attempt_id", "scenario", "profile", "timestamp_start", "timestamp_end",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
549	"battery_start", "battery_end", "height_start_cm", "height_end_cm",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
550	"obstacle_class_expected", "detection_success", "triggered", "avoidance_success",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
551	"false_trigger", "first_detection_frame", "trigger_frame", "clearance_frame",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
552	"trigger_area_norm", "trigger_growth", "trigger_growth_rate", "trigger_ttc_proxy_sec",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
553	"trigger_cx", "trigger_distance_est_m", "avoid_direction", "avoidance_mode",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
554	"vertical_duration_sec", "vertical_effort", "lateral_duration_sec",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
555	"lateral_effort", "bypass_duration_sec", "bypass_distance_est_m",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
556	"recenter_target_sec", "notes"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
557	]	Cierra una estructura o llamada multilinea.
558	)	Cierra una estructura o llamada multilinea.
559	detections_f, detections_writer = open_csv_writer(	Ejecuta una instruccion necesaria para el flujo del programa.
560	run_dir / "detections.csv",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
561	[	Ejecuta una instruccion necesaria para el flujo del programa.
562	"timestamp", "frame_idx", "attempt_id", "scenario", "profile", "state",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
563	"det_idx", "class_name", "confidence", "x1", "y1", "x2", "y2",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
564	"x1_norm", "x2_norm", "bbox_area_norm", "center_x_norm", "center_y_norm",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
565	"distance_est_m", "is_target", "is_primary", "intersects_safety_corridor",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
566	"y1_norm", "y2_norm", "bbox_height_norm", "below_up_clearance", "above_down_clearance"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
567	]	Cierra una estructura o llamada multilinea.
568	)	Cierra una estructura o llamada multilinea.
569	telemetry_f, telemetry_writer = open_csv_writer(	Ejecuta una instruccion necesaria para el flujo del programa.
570	run_dir / "telemetry.csv",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.

Line	Code	Explanation
571	[	Ejecuta una instruccion necesaria para el flujo del programa.
572	"timestamp", "frame_idx", "attempt_id", "scenario", "profile",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
573	"battery", "height_cm", "tof_cm", "yaw", "pitch", "roll",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
574	"vgx", "vgy", "vgz", "state", "flying", "attempt_active",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
575	"lateral_effort", "bypass_distance_est_m", "recenter_target_sec",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
576	"avoidance_mode", "vertical_effort", "vertical_duration_sec"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
577	]	Cierra una estructura o llamada multilinea.
578	)	Cierra una estructura o llamada multilinea.
579	labels_f, labels_writer = open_csv_writer(	Ejecuta una instruccion necesaria para el flujo del programa.
580	run_dir / "operator_trial_labels.csv",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
581	[	Ejecuta una instruccion necesaria para el flujo del programa.
582	"timestamp", "attempt_id", "scenario", "profile",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
583	"operator_label", "operator_success", "auto_avoidance_success",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
584	"triggered", "detection_success", "notes"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
585	]	Cierra una estructura o llamada multilinea.
586	)	Cierra una estructura o llamada multilinea.
587		Linea en blanco para separar bloques logicos del codigo.
588	def log_event(frame_idx, event, details=""):	Define la funcion log_event.
589	print(f"[EVENT] {event} {details}")	Muestra informacion por consola.
590	events_writer.writerow([now_iso(), frame_idx, event, details])	Escribe una fila en un archivo CSV de registro.
591	events_f.flush()	Fuerza el guardado inmediato de los datos pendientes.
592		Linea en blanco para separar bloques logicos del codigo.
593	tello = Tello()	Crea el objeto usado para controlar el dron Tello.
594	tello.connect()	Conecta el ordenador con el dron Tello mediante el SDK.
595	battery = tello.get_battery()	Calcula o inicializa la variable battery.
596	print(f"[INFO] Profile: {PROFILE_NAME}")	Muestra informacion por consola.
597	print(f"[INFO] Battery: {battery}%")	Muestra informacion por consola.
598	if battery < MIN_BATTERY_PERCENT:	Evalua una condicion antes de ejecutar el bloque siguiente.
599	raise RuntimeError(f"Battery too low for safe testing: {battery}%")	Detiene la ejecucion lanzando un error controlado.
600		Linea en blanco para separar bloques logicos del codigo.
601	tello.streamon()	Activa el flujo de video del dron.
602	frame_read = tello.get_frame_read()	Calcula o inicializa la variable frame_read.
603	time.sleep(1.5)	Ejecuta una instruccion necesaria para el flujo del programa.
604		Linea en blanco para separar bloques logicos del codigo.
605	raw_writer = None	Calcula o inicializa la variable raw_writer.
606	ann_writer = None	Calcula o inicializa la variable ann_writer.
607	frame_idx = 0	Inicializa frame_idx con un valor neutro o por defecto.
608	last_frame_save = 0.0	Inicializa last_frame_save con un valor neutro o por defecto.

Line	Code	Explanation
609	last_telemetry_log = 0.0	Inicializa last_telemetry_log con un valor neutro o por defecto.
610		Linea en blanco para separar bloques logicos del codigo.
611	flying = False	Inicializa la bandera booleana flying.
612	state = "IDLE"	Calcula o inicializa la variable state.
613	attempt_id = 0	Inicializa attempt_id con un valor neutro o por defecto.
614	attempt_active = False	Inicializa la bandera booleana attempt_active.
615	scenario_name = "centered_person"	Calcula o inicializa la variable scenario_name.
616	expected_obstacle_class = "person"	Calcula o inicializa la variable expected_obstacle_class.
617	attempt_start_ts = ""	Calcula o inicializa la variable attempt_start_ts.
618	attempt_start_time = 0.0	Inicializa attempt_start_time con un valor neutro o por defecto.
619	attempt_notes = ""	Calcula o inicializa la variable attempt_notes.
620	battery_start = -1	Inicializa battery_start con un valor neutro o por defecto.
621	height_start = -1	Inicializa height_start con un valor neutro o por defecto.
622	first_detection_frame = -1	Inicializa first_detection_frame con un valor neutro o por defecto.
623	trigger_frame = -1	Inicializa trigger_frame con un valor neutro o por defecto.
624	clearance_frame = -1	Inicializa clearance_frame con un valor neutro o por defecto.
625	trigger_area = 0.0	Inicializa trigger_area con un valor neutro o por defecto.
626	trigger_growth = 0.0	Inicializa trigger_growth con un valor neutro o por defecto.
627	trigger_growth_rate = 0.0	Inicializa trigger_growth_rate con un valor neutro o por defecto.
628	trigger_ttc_proxy = -1.0	Inicializa trigger_ttc_proxy con un valor neutro o por defecto.
629	trigger_cx = 0.0	Inicializa trigger_cx con un valor neutro o por defecto.
630	trigger_distance_est = -1.0	Inicializa trigger_distance_est con un valor neutro o por defecto.
631	detection_success = False	Inicializa la bandera booleana detection_success.
632	triggered = False	Inicializa la bandera booleana triggered.
633	avoidance_success = False	Inicializa la bandera booleana avoidance_success.
634	false_trigger = False	Inicializa la bandera booleana false_trigger.
635	avoid_direction_text = "none"	Calcula o inicializa la variable avoid_direction_text.
636	avoid_direction_sign = 0	Inicializa avoid_direction_sign con un valor neutro o por defecto.
637	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
638	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
639	recenter_clear_counter = 0	Inicializa recenter_clear_counter con un valor neutro o por defecto.
640	state_start_time = 0.0	Inicializa state_start_time con un valor neutro o por defecto.
641	lateral_start_time = 0.0	Inicializa lateral_start_time con un valor neutro o por defecto.
642	lateral_duration = 0.0	Inicializa lateral_duration con un valor neutro o por defecto.
643	bypass_start_time = 0.0	Inicializa bypass_start_time con un valor neutro o por defecto.
644	bypass_duration = 0.0	Inicializa bypass_duration con un valor neutro o por defecto.
645	bypass_distance_est_m = 0.0	Inicializa bypass_distance_est_m con un valor neutro o por defecto.
646	lateral_effort = 0.0	Inicializa lateral_effort con un valor neutro o por defecto.

Line	Code	Explanation
647	avoidance_mode = "none"	Calcula o inicializa la variable avoidance_mode.
648	up_start_time = 0.0	Inicializa up_start_time con un valor neutro o por defecto.
649	up_duration = 0.0	Inicializa up_duration con un valor neutro o por defecto.
650	vertical_effort = 0.0	Inicializa vertical_effort con un valor neutro o por defecto.
651	recenter_target_sec = RECENTER_SEC	Calcula o inicializa la variable recenter_target_sec.
652	last_motion_update_time = time.time()	Guarda el instante actual en last_motion_update_time.
653	smoothed_fb = 0	Inicializa smoothed_fb con un valor neutro o por defecto.
654	smoothed_lr = 0	Inicializa smoothed_lr con un valor neutro o por defecto.
655		Linea en blanco para separar bloques logicos del codigo.
656		Linea en blanco para separar bloques logicos del codigo.
657	awaiting_operator_label = False	Inicializa la bandera booleana awaiting_operator_label.
658	pending_label_attempt_id = -1	Inicializa pending_label_attempt_id con un valor neutro o por defecto.
659	pending_label_auto_success = False	Inicializa la bandera booleana pending_label_auto_success.
660	pending_label_triggered = False	Inicializa la bandera booleana pending_label_triggered.
661	pending_label_detection_success = False	Inicializa la bandera booleana pending_label_detection_success.
662	pending_label_notes = ""	Calcula o inicializa la variable pending_label_notes.
663	labelled_trials = 0	Inicializa labelled_trials con un valor neutro o por defecto.
664	labelled_successes = 0	Inicializa labelled_successes con un valor neutro o por defecto.
665		Linea en blanco para separar bloques logicos del codigo.
666	area_history = deque(maxlen=AREA_HISTORY_LEN)	Crea una cola limitada para almacenar valores recientes.
667	confirm_counter = 0	Inicializa confirm_counter con un valor neutro o por defecto.
668		Linea en blanco para separar bloques logicos del codigo.
669	def reset_smoothing():	Define la funcion reset_smoothing.
670	nonlocal smoothed_fb, smoothed_lr	Permite modificar variables definidas en la funcion exterior.
671	smoothed_fb = 0	Inicializa smoothed_fb con un valor neutro o por defecto.
672	smoothed_lr = 0	Inicializa smoothed_lr con un valor neutro o por defecto.
673		Linea en blanco para separar bloques logicos del codigo.
674	def stop_and_reset():	Define la funcion stop_and_reset.
675	safe_stop_rc(tello)	Detiene los comandos de movimiento del dron de forma segura.
676	reset_smoothing()	Reinicia los valores suavizados de control.
677		Linea en blanco para separar bloques logicos del codigo.
678	def send_smooth_rc(forward_backward_target, left_right_target, up_down_target=0, yaw_target=0):	Define la funcion send_smooth_rc.
679	nonlocal smoothed_fb, smoothed_lr	Permite modificar variables definidas en la funcion exterior.
680	smoothed_fb = smooth(smoothed_fb, forward_backward_target)	Calcula o inicializa la variable smoothed_fb.
681	smoothed_lr = smooth(smoothed_lr, left_right_target)	Calcula o inicializa la variable smoothed_lr.
682	tello.send_rc_control(int(smoothed_lr), int(smoothed_fb), int(up_down_target), int(yaw_target))	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
683		Linea en blanco para separar bloques logicos del codigo.
684	def set_state(new_state, reason=""):	Define la funcion set_state.

Line	Code	Explanation
685	nonlocal state, state_start_time, last_motion_update_time	Permite modificar variables definidas en la funcion exterior.
686	if state != new_state:	Evalua una condicion antes de ejecutar el bloque siguiente.
687	state = new_state	Calcula o inicializa la variable state.
688	state_start_time = time.time()	Guarda el instante actual en state_start_time.
689	last_motion_update_time = time.time()	Guarda el instante actual en last_motion_update_time.
690	log_event(frame_idx, f"state_{new_state}", reason)	Registra un evento relevante del experimento.
691		Linea en blanco para separar bloques logicos del codigo.
692	def reset_attempt_metrics():	Define la funcion reset_attempt_metrics.
693	nonlocal attempt_notes, first_detection_frame, trigger_frame, clearance_frame	Permite modificar variables definidas en la funcion exterior.
694	nonlocal trigger_area, trigger_growth, trigger_growth_rate, trigger_ttc_proxy	Permite modificar variables definidas en la funcion exterior.
695	nonlocal trigger_cx, trigger_distance_est	Permite modificar variables definidas en la funcion exterior.
696	nonlocal detection_success, triggered, avoidance_success, false_trigger	Permite modificar variables definidas en la funcion exterior.
697	nonlocal avoid_direction_text, avoid_direction_sign, exit_counter, lost_counter	Permite modificar variables definidas en la funcion exterior.
698	nonlocal recenter_clear_counter, state_start_time, lateral_start_time	Permite modificar variables definidas en la funcion exterior.
699	nonlocal lateral_duration, bypass_start_time, bypass_duration, confirm_counter	Permite modificar variables definidas en la funcion exterior.
700	nonlocal bypass_distance_est_m, lateral_effort, avoidance_mode, up_start_time, up_duration	Permite modificar variables definidas en la funcion exterior.
701	nonlocal vertical_effort, recenter_target_sec, last_motion_update_time	Permite modificar variables definidas en la funcion exterior.
702	attempt_notes = ""	Calcula o inicializa la variable attempt_notes.
703	first_detection_frame = -1	Inicializa first_detection_frame con un valor neutro o por defecto.
704	trigger_frame = -1	Inicializa trigger_frame con un valor neutro o por defecto.
705	clearance_frame = -1	Inicializa clearance_frame con un valor neutro o por defecto.
706	trigger_area = 0.0	Inicializa trigger_area con un valor neutro o por defecto.
707	trigger_growth = 0.0	Inicializa trigger_growth con un valor neutro o por defecto.
708	trigger_growth_rate = 0.0	Inicializa trigger_growth_rate con un valor neutro o por defecto.
709	trigger_ttc_proxy = -1.0	Inicializa trigger_ttc_proxy con un valor neutro o por defecto.
710	trigger_cx = 0.0	Inicializa trigger_cx con un valor neutro o por defecto.
711	trigger_distance_est = -1.0	Inicializa trigger_distance_est con un valor neutro o por defecto.
712	detection_success = False	Inicializa la bandera booleana detection_success.
713	triggered = False	Inicializa la bandera booleana triggered.
714	avoidance_success = False	Inicializa la bandera booleana avoidance_success.
715	false_trigger = False	Inicializa la bandera booleana false_trigger.
716	avoid_direction_text = "none"	Calcula o inicializa la variable avoid_direction_text.
717	avoid_direction_sign = 0	Inicializa avoid_direction_sign con un valor neutro o por defecto.
718	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
719	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
720	recenter_clear_counter = 0	Inicializa recenter_clear_counter con un valor neutro o por defecto.
721	area_history.clear()	Ejecuta una instruccion necesaria para el flujo del programa.
722	confirm_counter = 0	Inicializa confirm_counter con un valor neutro o por defecto.

Line	Code	Explanation
723	state_start_time = time.time()	Guarda el instante actual en state_start_time.
724	lateral_start_time = 0.0	Inicializa lateral_start_time con un valor neutro o por defecto.
725	lateral_duration = 0.0	Inicializa lateral_duration con un valor neutro o por defecto.
726	bypass_start_time = 0.0	Inicializa bypass_start_time con un valor neutro o por defecto.
727	bypass_duration = 0.0	Inicializa bypass_duration con un valor neutro o por defecto.
728	bypass_distance_est_m = 0.0	Inicializa bypass_distance_est_m con un valor neutro o por defecto.
729	lateral_effort = 0.0	Inicializa lateral_effort con un valor neutro o por defecto.
730	avoidance_mode = "none"	Calcula o inicializa la variable avoidance_mode.
731	up_start_time = 0.0	Inicializa up_start_time con un valor neutro o por defecto.
732	up_duration = 0.0	Inicializa up_duration con un valor neutro o por defecto.
733	vertical_effort = 0.0	Inicializa vertical_effort con un valor neutro o por defecto.
734	recenter_target_sec = RECENTER_SEC	Calcula o inicializa la variable recenter_target_sec.
735	last_motion_update_time = time.time()	Guarda el instante actual en last_motion_update_time.
736	reset_smoothing()	Reinicia los valores suavizados de control.
737		Línea en blanco para separar bloques lógicos del código.
738	def finalize_attempt(notes_suffix=""):	Define la función finalize_attempt.
739	nonlocal attempt_active, state, bypass_duration	Permite modificar variables definidas en la función exterior.
740	nonlocal awaiting_operator_label, pending_label_attempt_id, pending_label_auto_success	Permite modificar variables definidas en la función exterior.
741	nonlocal pending_label_triggered, pending_label_detection_success, pending_label_notes	Permite modificar variables definidas en la función exterior.
742	if not attempt_active:	Evalúa una condición antes de ejecutar el bloque siguiente.
743	return	Salida de la función sin devolver un valor específico.
744	if bypass_start_time > 0.0:	Evalúa una condición antes de ejecutar el bloque siguiente.
745	bypass_duration = max(bypass_duration, time.time() - bypass_start_time)	Calcula un valor máximo y lo asigna a bypass_duration.
746	notes = attempt_notes	Calcula o inicializa la variable notes.
747	if notes_suffix:	Evalúa una condición antes de ejecutar el bloque siguiente.
748	notes = f"{notes} {notes_suffix}" if notes else notes_suffix	Construye una cadena formateada y la asigna a notes.
749	attempts_writer.writerow([	Escribe una fila en un archivo CSV de registro.
750	attempt_id, scenario_name, PROFILE_NAME, attempt_start_ts, now_iso(),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
751	battery_start, safe_call(tello.get_battery),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
752	height_start, safe_call(tello.get_height),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
753	expected_obstacle_class,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
754	int(detection_success), int(triggered), int(avoidance_success),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
755	int(false_trigger), first_detection_frame, trigger_frame, clearance_frame,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
756	f"{trigger_area:.6f}", f"{trigger_growth:.6f}", f"{trigger_growth_rate:.6f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
757	f"{trigger_ttc_proxy:.3f}", f"{trigger_cx:.6f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
758	f"{trigger_distance_est:.3f}", avoid_direction_text, avoidance_mode,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
759	f"{up_duration:.3f}", f"{vertical_effort:.3f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
760	f"{lateral_duration:.3f}", f"{lateral_effort:.3f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.

Line	Code	Explanation
761	f"{bypass_duration:.3f}", f"{bypass_distance_est_m:.3f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
762	f"{recenter_target_sec:.3f}", notes	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
763	])	Cierra una expresion multilínea y continua el flujo del codigo.
764	attempts_f.flush()	Fuerza el guardado inmediato de los datos pendientes.
765	awaiting_operator_label = True	Inicializa la bandera booleana awaiting_operator_label.
766	pending_label_attempt_id = attempt_id	Calcula o inicializa la variable pending_label_attempt_id.
767	pending_label_auto_success = bool(avoidance_success)	Calcula o inicializa la variable pending_label_auto_success.
768	pending_label_triggered = bool(triggered)	Calcula o inicializa la variable pending_label_triggered.
769	pending_label_detection_success = bool(detection_success)	Calcula o inicializa la variable pending_label_detection_success.
770	pending_label_notes = notes	Calcula o inicializa la variable pending_label_notes.
771	attempt_active = False	Inicializa la bandera booleana attempt_active.
772	state = "HOVER" if flying else "IDLE"	Calcula o inicializa la variable state.
773	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
774	log_event(frame_idx, "attempt_finalized", f"id={attempt_id}, success={avoidance_success}, notes={notes}")	Registra un evento relevante del experimento.
775	log_event(frame_idx, "operator_label_required", "Press Y=success, F=failure, U=unclear/invalid before next attempt")	Registra un evento relevante del experimento.
776		Linea en blanco para separar bloques logicos del codigo.
777	def start_attempt():	Define la funcion start_attempt.
778	nonlocal attempt_id, attempt_active, attempt_start_ts, attempt_start_time	Permite modificar variables definidas en la funcion exterior.
779	nonlocal battery_start, height_start, state, state_start_time, last_motion_update_time	Permite modificar variables definidas en la funcion exterior.
780	nonlocal awaiting_operator_label	Permite modificar variables definidas en la funcion exterior.
781	if awaiting_operator_label:	Evalua una condicion antes de ejecutar el bloque siguiente.
782	log_event(frame_idx, "start_blocked", "label_previous_attempt_first_Y_F_U")	Registra un evento relevante del experimento.
783	return	Sale de la funcion sin devolver un valor especifico.
784	if not flying or attempt_active:	Evalua una condicion antes de ejecutar el bloque siguiente.
785	return	Sale de la funcion sin devolver un valor especifico.
786	attempt_id += 1	Ejecuta una instruccion necesaria para el flujo del programa.
787	attempt_start_ts = now_iso()	Guarda una marca temporal ISO en attempt_start_ts.
788	attempt_start_time = time.time()	Guarda el instante actual en attempt_start_time.
789	battery_start = safe_call(tello.get_battery)	Obtiene un valor de telemetria de forma segura y lo guarda en battery_start.
790	height_start = safe_call(tello.get_height)	Obtiene un valor de telemetria de forma segura y lo guarda en height_start.
791	reset_attempt_metrics()	Ejecuta una instruccion necesaria para el flujo del programa.
792	attempt_active = True	Inicializa la bandera booleana attempt_active.
793	state = "APPROACH_FORWARD"	Calcula o inicializa la variable state.
794	state_start_time = time.time()	Guarda el instante actual en state_start_time.
795	last_motion_update_time = time.time()	Guarda el instante actual en last_motion_update_time.
796	log_event(frame_idx, "attempt_started", f"id={attempt_id}, scenario={scenario_name}, profile={PROFILE_NAME}")	Registra un evento relevante del experimento.
797		Linea en blanco para separar bloques logicos del codigo.
798	def label_last_attempt(operator_label, operator_success, note=""):	Define la funcion label_last_attempt.

Line	Code	Explanation
799	nonlocal awaiting_operator_label, labelled_trials, labelled_successes	Permite modificar variables definidas en la funcion exterior.
800	if not awaiting_operator_label:	Evalua una condicion antes de ejecutar el bloque siguiente.
801	return	Sale de la funcion sin devolver un valor especifico.
802	labels_writer.writerow([	Escribe una fila en un archivo CSV de registro.
803	now_iso(), pending_label_attempt_id, scenario_name, PROFILE_NAME,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
804	operator_label, int(operator_success) if operator_success is not None else -1,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
805	int(pending_label_auto_success), int(pending_label_triggered),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
806	int(pending_label_detection_success),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
807	f"{pending_label_notes} {note}" if note else pending_label_notes	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
808	])	Cierra una expresion multilinea y continua el flujo del codigo.
809	labels_f.flush()	Fuerza el guardado inmediato de los datos pendientes.
810	labelled_trials += 1	Ejecuta una instruccion necesaria para el flujo del programa.
811	if operator_success is True:	Evalua una condicion antes de ejecutar el bloque siguiente.
812	labelled_successes += 1	Ejecuta una instruccion necesaria para el flujo del programa.
813	awaiting_operator_label = False	Inicializa la bandera booleana awaiting_operator_label.
814	log_event(frame_idx, "operator_label_saved", f"attempt={pending_label_attempt_id}, label={operator_label}")	Registra un evento relevante del experimento.
815		Linea en blanco para separar bloques logicos del codigo.
816	try:	Inicia un bloque protegido frente a errores de ejecucion.
817	while True:	Inicia el bucle principal de ejecucion continua.
818	frame = frame_read.frame	Calcula o inicializa la variable frame.
819	if frame is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
820	continue	Salta al siguiente ciclo del bucle actual.
821	frame = frame.copy()	Calcula o inicializa la variable frame.
822	raw_frame = frame.copy()	Calcula o inicializa la variable raw_frame.
823	H, W = frame.shape[:2]	Ejecuta una instruccion necesaria para el flujo del programa.
824	timestamp = now_iso()	Guarda una marca temporal ISO en timestamp.
825	now_t = time.time()	Guarda el instante actual en now_t.
826		Linea en blanco para separar bloques logicos del codigo.
827	if ann_writer is None and SAVE_ANNOTATED_VIDEO:	Evalua una condicion antes de ejecutar el bloque siguiente.
828	ann_writer = cv2.VideoWriter(	Crea el escritor de video asociado a ann_writer.
829	str(run_dir / "video_annotated.mp4"),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
830	cv2.VideoWriter_fourcc(*"mp4v"),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
831	VIDEO_FPS,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
832	(W, H)	Ejecuta una instruccion necesaria para el flujo del programa.
833	)	Cierra una estructura o llamada multilinea.
834	if raw_writer is None and SAVE_RAW_VIDEO:	Evalua una condicion antes de ejecutar el bloque siguiente.
835	raw_writer = cv2.VideoWriter(	Crea el escritor de video asociado a raw_writer.
836	str(run_dir / "video_raw.mp4"),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.

Line	Code	Explanation
837	cv2.VideoWriter_fourcc(*"mp4v"),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
838	VIDEO_FPS,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
839	(W, H)	Ejecuta una instruccion necesaria para el flujo del programa.
840	)	Cierra una estructura o llamada multilinea.
841		Linea en blanco para separar bloques logicos del codigo.
842		Linea en blanco para separar bloques logicos del codigo.
843	# Perception	Comentario corto que identifica la seccion del codigo.
844	results = model.predict(source=frame, conf=CONF_THRES, verbose=False)[0]	Calcula o inicializa la variable results.
845	detections = []	Calcula o inicializa la variable detections.
846	primary = None	Calcula o inicializa la variable primary.
847		Linea en blanco para separar bloques logicos del codigo.
848	if results.bboxes is not None and len(results.bboxes) > 0:	Evalua una condicion antes de ejecutar el bloque siguiente.
849	for det_idx, box in enumerate(results.bboxes):	Inicia un bucle sobre una secuencia de elementos.
850	cls_id = int(box.cls[0].item())	Convierte el resultado a entero y lo guarda en cls_id.
851	cls_name = model.names[cls_id]	Calcula o inicializa la variable cls_name.
852	conf = float(box.conf[0].item())	Convierte el resultado a flotante y lo guarda en conf.
853	x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())	Ejecuta una instruccion necesaria para el flujo del programa.
854	bw = max(0, x2 - x1)	Calcula un valor maximo y lo asigna a bw.
855	bh = max(0, y2 - y1)	Calcula un valor maximo y lo asigna a bh.
856	area_norm = (bw * bh) / float(W * H)	Calcula o inicializa la variable area_norm.
857	cx = ((x1 + x2) / 2.0) / W	Calcula o inicializa la variable cx.
858	cy = ((y1 + y2) / 2.0) / H	Calcula o inicializa la variable cy.
859	x1n = x1 / W	Calcula o inicializa la variable x1n.
860	x2n = x2 / W	Calcula o inicializa la variable x2n.
861	y1n = y1 / H	Calcula o inicializa la variable y1n.
862	y2n = y2 / H	Calcula o inicializa la variable y2n.
863	distance_est_det = estimate_distance_from_area(area_norm)	Calcula o inicializa la variable distance_est_det.
864	is_target = cls_name in TARGET_CLASSES	Calcula o inicializa la variable is_target.
865	detections.append({	Anade un dato a una lista o cola.
866	"class_name": cls_name,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
867	"confidence": conf,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
868	"x1": x1, "y1": y1, "x2": x2, "y2": y2,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
869	"x1_norm": x1n, "x2_norm": x2n,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
870	"y1_norm": y1n, "y2_norm": y2n,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
871	"bbox_area_norm": area_norm,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
872	"center_x_norm": cx,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
873	"center_y_norm": cy,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
874	"distance_est_m": distance_est_det,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.

Line	Code	Explanation
875	"is_target": is_target,	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
876	}}	Cierra una expresion multilínea y continua el flujo del codigo.
877		Linea en blanco para separar bloques logicos del codigo.
878	target_dets = [d for d in detections if d["is_target"]]	Calcula o inicializa la variable target_dets.
879	if target_dets:	Evalua una condicion antes de ejecutar el bloque siguiente.
880	primary = max(target_dets, key=lambda d: d["bbox_area_norm"])	Calcula un valor maximo y lo asigna a primary.
881		Linea en blanco para separar bloques logicos del codigo.
882		Linea en blanco para separar bloques logicos del codigo.
883	# Decision	Comentario corto que identifica la seccion del codigo.
884	current_area = 0.0	Inicializa current_area con un valor neutro o por defecto.
885	current_growth = 0.0	Inicializa current_growth con un valor neutro o por defecto.
886	current_growth_rate = 0.0	Inicializa current_growth_rate con un valor neutro o por defecto.
887	current_cx = 0.5	Calcula o inicializa la variable current_cx.
888	distance_est = None	Calcula o inicializa la variable distance_est.
889	ttc_proxy = None	Calcula o inicializa la variable ttc_proxy.
890	danger = False	Inicializa la bandera booleana danger.
891	warning = False	Inicializa la bandera booleana warning.
892	hard_stop = False	Inicializa la bandera booleana hard_stop.
893	centered = False	Inicializa la bandera booleana centered.
894	corridor_overlap = False	Inicializa la bandera booleana corridor_overlap.
895		Linea en blanco para separar bloques logicos del codigo.
896	if primary is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
897	current_area = primary["bbox_area_norm"]	Calcula o inicializa la variable current_area.
898	current_cx = primary["center_x_norm"]	Calcula o inicializa la variable current_cx.
899	distance_est = primary["distance_est_m"]	Calcula o inicializa la variable distance_est.
900	area_history.append((now_t, current_area))	Anade un dato a una lista o cola.
901	if len(area_history) >= 2:	Evalua una condicion antes de ejecutar el bloque siguiente.
902	current_growth = area_history[-1][1] - area_history[0][1]	Calcula o inicializa la variable current_growth.
903	current_growth_rate = compute_area_growth_rate(area_history)	Calcula o inicializa la variable current_growth_rate.
904	ttc_proxy = compute_ttc_proxy(current_area, current_growth_rate)	Calcula o inicializa la variable ttc_proxy.
905		Linea en blanco para separar bloques logicos del codigo.
906	centered = CENTER_X_MIN <= current_cx <= CENTER_X_MAX	Calcula o inicializa la variable centered.
907	corridor_overlap = bbox_intersects_safety_corridor(primary)	Calcula o inicializa la variable corridor_overlap.
908	warning = corridor_overlap and current_area >= AREA_WARNING	Calcula o inicializa la variable warning.
909		Linea en blanco para separar bloques logicos del codigo.
910	area_hard_stop = corridor_overlap and current_area >= AREA_HARD_STOP	Calcula o inicializa la variable area_hard_stop.
911	distance_hard_stop = (	Calcula o inicializa la variable distance_hard_stop.
912	USE_DISTANCE_TRIGGER and	Ejecuta una instruccion necesaria para el flujo del programa.

Line	Code	Explanation
913	distance_est is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
914	distance_est <= HARD_STOP_DISTANCE_M and	Ejecuta una instruccion necesaria para el flujo del programa.
915	corridor_overlap	Ejecuta una instruccion necesaria para el flujo del programa.
916	)	Cierra una estructura o llamada multilinea.
917	hard_stop = area_hard_stop or distance_hard_stop	Calcula o inicializa la variable hard_stop.
918		Linea en blanco para separar bloques logicos del codigo.
919	area_danger_candidate = corridor_overlap and (	Calcula o inicializa la variable area_danger_candidate.
920	current_area >= AREA_TRIGGER or	Ejecuta una instruccion necesaria para el flujo del programa.
921	(current_area >= AREA_WARNING and current_growth >= GROWTH_TRIGGER)	Ejecuta una instruccion necesaria para el flujo del programa.
922	)	Cierra una estructura o llamada multilinea.
923	distance_danger_candidate = (	Calcula o inicializa la variable distance_danger_candidate.
924	USE_DISTANCE_TRIGGER and	Ejecuta una instruccion necesaria para el flujo del programa.
925	distance_est is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
926	distance_est <= TRIGGER_DISTANCE_M and	Ejecuta una instruccion necesaria para el flujo del programa.
927	corridor_overlap	Ejecuta una instruccion necesaria para el flujo del programa.
928	)	Cierra una estructura o llamada multilinea.
929	ttc_danger_candidate = (	Calcula o inicializa la variable ttc_danger_candidate.
930	USE_TTC_TRIGGER and	Ejecuta una instruccion necesaria para el flujo del programa.
931	ttc_proxy is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
932	current_area >= MIN_TTC_AREA and	Ejecuta una instruccion necesaria para el flujo del programa.
933	current_growth_rate >= MIN_AREA_GROWTH_RATE and	Ejecuta una instruccion necesaria para el flujo del programa.
934	ttc_proxy <= TTC_TRIGGER_SEC and	Ejecuta una instruccion necesaria para el flujo del programa.
935	corridor_overlap	Ejecuta una instruccion necesaria para el flujo del programa.
936	)	Cierra una estructura o llamada multilinea.
937	up_danger_candidate = (	Calcula o inicializa la variable up_danger_candidate.
938	ENABLE_UPWARD_AVOIDANCE and	Ejecuta una instruccion necesaria para el flujo del programa.
939	primary["class_name"] in UP_ONLY_FOR_CLASSES and	Ejecuta una instruccion necesaria para el flujo del programa.
940	corridor_overlap and	Ejecuta una instruccion necesaria para el flujo del programa.
941	(	Ejecuta una instruccion necesaria para el flujo del programa.
942	current_area >= UP_AREA_TRIGGER or	Ejecuta una instruccion necesaria para el flujo del programa.
943	(distance_est is not None and distance_est <= UP_DISTANCE_TRIGGER_M)	Ejecuta una instruccion necesaria para el flujo del programa.
944	)	Cierra una estructura o llamada multilinea.
945	)	Cierra una estructura o llamada multilinea.
946	down_danger_candidate = (	Calcula o inicializa la variable down_danger_candidate.
947	ENABLE_DOWNWARD_AVOIDANCE and	Ejecuta una instruccion necesaria para el flujo del programa.
948	primary["class_name"] in DOWN_ONLY_FOR_CLASSES and	Ejecuta una instruccion necesaria para el flujo del programa.
949	corridor_overlap and	Ejecuta una instruccion necesaria para el flujo del programa.
950	(	Ejecuta una instruccion necesaria para el flujo del programa.

Line	Code	Explanation
951	current_area >= DOWN_AREA_TRIGGER or	Ejecuta una instruccion necesaria para el flujo del programa.
952	(distance_est is not None and distance_est <= DOWN_DISTANCE_TRIGGER_M)	Ejecuta una instruccion necesaria para el flujo del programa.
953	)	Cierra una estructura o llamada multilinea.
954	)	Cierra una estructura o llamada multilinea.
955	danger_candidate = (	Calcula o inicializa la variable danger_candidate.
956	area_danger_candidate or	Ejecuta una instruccion necesaria para el flujo del programa.
957	distance_danger_candidate or	Ejecuta una instruccion necesaria para el flujo del programa.
958	ttc_danger_candidate or	Ejecuta una instruccion necesaria para el flujo del programa.
959	up_danger_candidate or	Ejecuta una instruccion necesaria para el flujo del programa.
960	down_danger_candidate	Ejecuta una instruccion necesaria para el flujo del programa.
961	)	Cierra una estructura o llamada multilinea.
962	confirm_counter = confirm_counter + 1 if danger_candidate else 0	Calcula o inicializa la variable confirm_counter.
963	danger = confirm_counter >= CONFIRM_FRAMES_REQUIRED	Calcula o inicializa la variable danger.
964		Linea en blanco para separar bloques logicos del codigo.
965	if attempt_active and first_detection_frame < 0 and primary["class_name"] == expected_obstacle_class:	Evalua una condicion antes de ejecutar el bloque siguiente.
966	first_detection_frame = frame_idx	Calcula o inicializa la variable first_detection_frame.
967	detection_success = True	Inicializa la bandera booleana detection_success.
968	log_event(frame_idx, "first_detection", f"attempt={attempt_id}, class={primary['class_name']}")	Registra un evento relevante del experimento.
969	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
970	area_history.append((now_t, 0.0))	Anade un dato a una lista o cola.
971	confirm_counter = 0	Inicializa confirm_counter con un valor neutro o por defecto.
972		Linea en blanco para separar bloques logicos del codigo.
973		Linea en blanco para separar bloques logicos del codigo.
974	for det_i, d in enumerate(detections):	Inicia un bucle sobre una secuencia de elementos.
975	if not d["is_target"] and not SHOW_NON_TARGETS:	Evalua una condicion antes de ejecutar el bloque siguiente.
976	continue	Salta al siguiente ciclo del bucle actual.
977	is_primary = primary is not None and d is primary	Calcula o inicializa la variable is_primary.
978	intersects = bbox_intersects_safety_corridor(d)	Calcula o inicializa la variable intersects.
979	color = (0, 255, 0) if d["is_target"] else (160, 160, 160)	Calcula o inicializa la variable color.
980	if is_primary:	Evalua una condicion antes de ejecutar el bloque siguiente.
981	color = (0, 0, 255)	Calcula o inicializa la variable color.
982	cv2.rectangle(frame, (d["x1"], d["y1"]), (d["x2"], d["y2"]), color, 2)	Dibuja un rectangulo sobre el frame.
983	dist_txt = ""	Calcula o inicializa la variable dist_txt.
984	if d["distance_est_m"] is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
985	dist_txt = f" d-{d['distance_est_m']:.2f}m"	Construye una cadena formateada y la asigna a dist_txt.
986	label = f"{d['class_name']} {d['confidence']:.2f} A={d['bbox_area_norm']:.3f}{dist_txt}"	Construye una cadena formateada y la asigna a label.
987	cv2.putText(frame, label, (d["x1"], max(20, d["y1"] - 8)), cv2.FONT_HERSHEY_SIMPLEX, 0.50, color, 2)	Escribe texto informativo sobre el frame.
988	detections_writer.writerow([	Escribe una fila en un archivo CSV de registro.

Line	Code	Explanation
989	timestamp, frame_idx, attempt_id if attempt_active else -1, scenario_name, PROFILE_NAME, state,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
990	det_i, d["class_name"], f"{d['confidence']:.4f}",	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
991	d["x1"], d["y1"], d["x2"], d["y2"],	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
992	f"{d['x1_norm']:.6f}", f"{d['x2_norm']:.6f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
993	f"{d['bbox_area_norm']:.6f}", f"{d['center_x_norm']:.6f}", f"{d['center_y_norm']:.6f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
994	f"{d['distance_est_m']:.3f}" if d["distance_est_m"] is not None else "",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
995	int(d["is_target"]), int(is_primary), int(intersects),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
996	f"{d.get('y1_norm', 0.0):.6f}", f"{d.get('y2_norm', 0.0):.6f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
997	f"{bbox_height_norm(d):.6f}", int(obstacle_below_vertical_clearance(d)),	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
998	int(obstacle_above_vertical_clearance(d))	Ejecuta una instruccion necesaria para el flujo del programa.
999	])	Cierra una expresion multilinea y continua el flujo del codigo.
1000	detections_f.flush()	Fuerza el guardado inmediato de los datos pendientes.
1001		Linea en blanco para separar bloques logicos del codigo.
1002		Linea en blanco para separar bloques logicos del codigo.
1003	# Actuation	Comentario corto que identifica la seccion del codigo.
1004	if attempt_active and flying:	Evalua una condicion antes de ejecutar el bloque siguiente.
1005	if (time.time() - attempt_start_time > MAX_TRIGGER_WAIT_SEC) and not triggered:	Evalua una condicion antes de ejecutar el bloque siguiente.
1006	log_event(frame_idx, "attempt_timeout", f"id={attempt_id}, no_trigger")	Registra un evento relevante del experimento.
1007	finalize_attempt("timeout_before_trigger")	Cierra el intento actual y guarda sus metricas.
1008		Linea en blanco para separar bloques logicos del codigo.
1009	if attempt_active and state == "APPROACH_FORWARD":	Evalua una condicion antes de ejecutar el bloque siguiente.
1010	approach_elapsed = time.time() - state_start_time	Guarda el instante actual en approach_elapsed.
1011		Linea en blanco para separar bloques logicos del codigo.
1012	if hard_stop:	Evalua una condicion antes de ejecutar el bloque siguiente.
1013	send_smooth_rc(EMERGENCY_BACK_SPEED, 0)	Envia un comando RC aplicando suavizado.
1014		Linea en blanco para separar bloques logicos del codigo.
1015	elif danger and primary is not None and approach_elapsed >= MIN_APPROACH_SEC:	Evalua una condicion alternativa dentro de la misma decision.
1016	triggered = True	Inicializa la bandera booleana triggered.
1017	trigger_frame = frame_idx	Calcula o inicializa la variable trigger_frame.
1018	trigger_area = current_area	Calcula o inicializa la variable trigger_area.
1019	trigger_growth = current_growth	Calcula o inicializa la variable trigger_growth.
1020	trigger_growth_rate = current_growth_rate	Calcula o inicializa la variable trigger_growth_rate.
1021	trigger_ttc_proxy = ttc_proxy if ttc_proxy is not None else -1.0	Calcula o inicializa la variable trigger_ttc_proxy.
1022	trigger_cx = current_cx	Calcula o inicializa la variable trigger_cx.
1023	trigger_distance_est = distance_est if distance_est is not None else -1.0	Calcula o inicializa la variable trigger_distance_est.
1024	current_height_cm = safe_call(tello.get_height)	Obtiene un valor de telemetria de forma segura y lo guarda en current_height_cm.
1025	avoidance_mode, avoid_direction_sign, avoid_direction_text, strategy_reason = choose_avoidance_strategy(	Ejecuta una instruccion necesaria para el flujo del programa.
1026	primary, current_area, distance_est, current_height_cm	Ejecuta una instruccion necesaria para el flujo del programa.

Line	Code	Explanation
1027	)	Cierra una estructura o llamada multilínea.
1028	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1029	set_state(	Cambia el estado de la maquina de estados y lo registra.
1030	"BRAKE_BEFORE_AVOID",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1031	f"area={current_area:.4f}, growth={current_growth:.4f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1032	f"growth_rate={current_growth_rate:.4f}, ttc={trigger_ttc_proxy:.2f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1033	f"dist={trigger_distance_est:.2f}, mode={avoidance_mode}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1034	f"dir={avoid_direction_text}, {strategy_reason}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1035	)	Cierra una estructura o llamada multilínea.
1036	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1037	send_smooth_rc(RC_APPROACH_FORWARD, 0)	Envía un comando RC aplicando suavizado.
1038		Linea en blanco para separar bloques logicos del codigo.
1039	elif attempt_active and state == "BRAKE_BEFORE_AVOID":	Evalua una condicion alternativa dentro de la misma decision.
1040	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1041	if time.time() - state_start_time >= BRAKE_BEFORE_AVOID_SEC:	Evalua una condicion antes de ejecutar el bloque siguiente.
1042	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1043	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1044	if avoidance_mode == "up":	Evalua una condicion antes de ejecutar el bloque siguiente.
1045	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1046	set_state("AVOID_UP_UNTIL_CLEAR", f"dir=up, target_sec={UP_AVOID_TARGET_SEC:.2f}")	Cambia el estado de la maquina de estados y lo registra.
1047	elif avoidance_mode == "down":	Evalua una condicion alternativa dentro de la misma decision.
1048	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1049	set_state("AVOID_DOWN_UNTIL_CLEAR", f"dir=down, target_sec={DOWN_AVOID_TARGET_SEC:.2f}")	Cambia el estado de la maquina de estados y lo registra.
1050	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1051	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1052	set_state("AVOID_LATERAL_UNTIL_EXIT", f"dir={avoid_direction_text}")	Cambia el estado de la maquina de estados y lo registra.
1053		Linea en blanco para separar bloques logicos del codigo.
1054	elif attempt_active and state == "AVOID_UP_UNTIL_CLEAR":	Evalua una condicion alternativa dentro de la misma decision.
1055	elapsed = time.time() - up_start_time	Guarda el instante actual en elapsed.
1056		Linea en blanco para separar bloques logicos del codigo.
1057	if primary is not None and obstacle_below_vertical_clearance(primary):	Evalua una condicion antes de ejecutar el bloque siguiente.
1058	exit_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1059	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1060	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1061		Linea en blanco para separar bloques logicos del codigo.
1062	if primary is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1063	lost_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1064	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.

Line	Code	Explanation
1065	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1066		Linea en blanco para separar bloques logicos del codigo.
1067	up_clear_confirmed = elapsed >= UP_AVOID_MIN_SEC and exit_counter >= EXIT_CORRIDOR_FRAMES_REQUIRED	Calcula o inicializa la variable up_clear_confirmed.
1068	lost_confirmed = elapsed >= UP_AVOID_MIN_SEC and lost_counter >= LOST_CLEAR_FRAMES_REQUIRED	Calcula o inicializa la variable lost_confirmed.
1069	target_lift_completed = elapsed >= UP_AVOID_TARGET_SEC	Calcula o inicializa la variable target_lift_completed.
1070		Linea en blanco para separar bloques logicos del codigo.
1071	if up_clear_confirmed or lost_confirmed or target_lift_completed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1072	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1073	clearance_frame = frame_idx	Calcula o inicializa la variable clearance_frame.
1074	up_duration = elapsed	Calcula o inicializa la variable up_duration.
1075	recenter_target_sec = DOWN_RECENTER_SEC if USE_DOWN_RECENTER_AFTER_UP else 0.0	Calcula o inicializa la variable recenter_target_sec.
1076	bypass_distance_est_m = 0.0	Inicializa bypass_distance_est_m con un valor neutro o por defecto.
1077	bypass_start_time = time.time()	Guarda el instante actual en bypass_start_time.
1078	if up_clear_confirmed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1079	reason = "bbox_below_vertical_clearance"	Calcula o inicializa la variable reason.
1080	elif lost_confirmed:	Evalua una condicion alternativa dentro de la misma decision.
1081	reason = "object_lost_after_vertical_motion"	Calcula o inicializa la variable reason.
1082	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1083	reason = "target_lift_time_completed"	Calcula o inicializa la variable reason.
1084	set_state(	Cambia el estado de la maquina de estados y lo registra.
1085	"FORWARD_AROUND_OBSTACLE",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1086	f"vertical_{reason}, up_sec={elapsed:.2f}, vertical_effort={vertical_effort:.1f}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1087	)	Cierra una estructura o llamada multilinea.
1088	elif elapsed >= UP_AVOID_MAX_SEC:	Evalua una condicion alternativa dentro de la misma decision.
1089	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1090	up_duration = elapsed	Calcula o inicializa la variable up_duration.
1091	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1092	set_state("AVOID_LATERAL_UNTIL_EXIT", "vertical_max_time_reached_fallback_to_lateral")	Cambia el estado de la maquina de estados y lo registra.
1093	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1094	motion_now = time.time()	Guarda el instante actual en motion_now.
1095	dt_motion = min(max(motion_now - last_motion_update_time, 0.0), 0.20)	Calcula un valor minimo y lo asigna a dt_motion.
1096	last_motion_update_time = motion_now	Calcula o inicializa la variable last_motion_update_time.
1097	vertical_effort += abs(RC_UP_AVOID) * dt_motion	Ejecuta una instruccion necesaria para el flujo del programa.
1098	send_smooth_rc(0, 0, RC_UP_AVOID)	Envia un comando RC aplicando suavizado.
1099		Linea en blanco para separar bloques logicos del codigo.
1100	elif attempt_active and state == "AVOID_DOWN_UNTIL_CLEAR":	Evalua una condicion alternativa dentro de la misma decision.
1101	elapsed = time.time() - up_start_time	Guarda el instante actual en elapsed.
1102		Linea en blanco para separar bloques logicos del codigo.

Line	Code	Explanation
1103	if primary is not None and obstacle_above_vertical_clearance(primary):	Evalua una condicion antes de ejecutar el bloque siguiente.
1104	exit_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1105	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1106	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1107		Linea en blanco para separar bloques logicos del codigo.
1108	if primary is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1109	lost_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1110	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1111	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1112		Linea en blanco para separar bloques logicos del codigo.
1113	down_clear_confirmed = elapsed >= DOWN_AVOID_MIN_SEC and exit_counter >= EXIT_CORRIDOR_FRAMES_REQUIRED	Calcula o inicializa la variable down_clear_confirmed.
1114	lost_confirmed = elapsed >= DOWN_AVOID_MIN_SEC and lost_counter >= LOST_CLEAR_FRAMES_REQUIRED	Calcula o inicializa la variable lost_confirmed.
1115	target_drop_completed = elapsed >= DOWN_AVOID_TARGET_SEC	Calcula o inicializa la variable target_drop_completed.
1116		Linea en blanco para separar bloques logicos del codigo.
1117	if down_clear_confirmed or lost_confirmed or target_drop_completed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1118	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1119	clearance_frame = frame_idx	Calcula o inicializa la variable clearance_frame.
1120	up_duration = elapsed	Calcula o inicializa la variable up_duration.
1121	recenter_target_sec = UP_RECENTER_AFTER_DOWN_SEC if USE_UP_RECENTER_AFTER_DOWN else 0.0	Calcula o inicializa la variable recenter_target_sec.
1122	bypass_distance_est_m = 0.0	Inicializa bypass_distance_est_m con un valor neutro o por defecto.
1123	bypass_start_time = time.time()	Guarda el instante actual en bypass_start_time.
1124	if down_clear_confirmed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1125	reason = "bbox_above_downward_clearance"	Calcula o inicializa la variable reason.
1126	elif lost_confirmed:	Evalua una condicion alternativa dentro de la misma decision.
1127	reason = "object_lost_after_downward_motion"	Calcula o inicializa la variable reason.
1128	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1129	reason = "target_drop_time_completed"	Calcula o inicializa la variable reason.
1130	set_state(	Cambia el estado de la maquina de estados y lo registra.
1131	"FORWARD_AROUND_OBSTACLE",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1132	f"vertical_{reason}, down_sec={elapsed:.2f}, vertical_effort={vertical_effort:.1f}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1133	)	Cierra una estructura o llamada multilinea.
1134	elif elapsed >= DOWN_AVOID_MAX_SEC:	Evalua una condicion alternativa dentro de la misma decision.
1135	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1136	up_duration = elapsed	Calcula o inicializa la variable up_duration.
1137	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1138	set_state("AVOID_LATERAL_UNTIL_EXIT", "downward_max_time_reached_fallback_to_lateral")	Cambia el estado de la maquina de estados y lo registra.
1139	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1140	motion_now = time.time()	Guarda el instante actual en motion_now.

Line	Code	Explanation
1141	dt_motion = min(max(motion_now - last_motion_update_time, 0.0), 0.20)	Calcula un valor minimo y lo asigna a dt_motion.
1142	last_motion_update_time = motion_now	Calcula o inicializa la variable last_motion_update_time.
1143	vertical_effort += abs(RC_DOWN_AVOID) * dt_motion	Ejecuta una instruccion necesaria para el flujo del programa.
1144	send_smooth_rc(0, 0, -RC_DOWN_AVOID)	Envia un comando RC aplicando suavizado.
1145		Linea en blanco para separar bloques logicos del codigo.
1146	elif attempt_active and state == "AVOID_LATERAL_UNTIL_EXIT":	Evalua una condicion alternativa dentro de la misma decision.
1147	elapsed = time.time() - lateral_start_time	Guarda el instante actual en elapsed.
1148		Linea en blanco para separar bloques logicos del codigo.
1149	if primary is not None and bbox_outside_safety_corridor(primary):	Evalua una condicion antes de ejecutar el bloque siguiente.
1150	exit_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1151	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1152	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1153		Linea en blanco para separar bloques logicos del codigo.
1154	if primary is None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1155	lost_counter += 1	Ejecuta una instruccion necesaria para el flujo del programa.
1156	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1157	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1158		Linea en blanco para separar bloques logicos del codigo.
1159	exit_confirmed = elapsed >= MIN_LATERAL_SEC and exit_counter >= EXIT_CORRIDOR_FRAMES_REQUIRED	Calcula o inicializa la variable exit_confirmed.
1160	lost_confirmed = elapsed >= MIN_LATERAL_SEC and lost_counter >= LOST_CLEAR_FRAMES_REQUIRED	Calcula o inicializa la variable lost_confirmed.
1161		Linea en blanco para separar bloques logicos del codigo.
1162	if exit_confirmed or lost_confirmed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1163	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1164	clearance_frame = frame_idx	Calcula o inicializa la variable clearance_frame.
1165	lateral_duration = elapsed	Calcula o inicializa la variable lateral_duration.
1166		Linea en blanco para separar bloques logicos del codigo.
1167	if USE_EFFORT_BASED_RECENTER:	Evalua una condicion antes de ejecutar el bloque siguiente.
1168	recenter_target_sec = (lateral_effort / max(RC_RECENTER, 1)) * RECENTER_COMPENSATION	Calcula o inicializa la variable recenter_target_sec.
1169	recenter_target_sec = max(MIN_RECENTER_SEC, min(MAX_RECENTER_SEC, recenter_target_sec))	Calcula un valor maximo y lo asigna a recenter_target_sec.
1170	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1171	recenter_target_sec = RECENTER_SEC	Calcula o inicializa la variable recenter_target_sec.
1172		Linea en blanco para separar bloques logicos del codigo.
1173	bypass_distance_est_m = 0.0	Inicializa bypass_distance_est_m con un valor neutro o por defecto.
1174	bypass_start_time = time.time()	Guarda el instante actual en bypass_start_time.
1175	reason = "bbox_outside_safety_corridor" if exit_confirmed else "object_lost_after_multiple_frames"	Calcula o inicializa la variable reason.
1176	set_state(	Cambia el estado de la maquina de estados y lo registra.
1177	"FORWARD_AROUND_OBSTACLE",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1178	f"{reason}, lateral_sec={elapsed:.2f}, lateral_effort={lateral_effort:.1f}, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.

Line	Code	Explanation
1179	f"recenter_target={recenter_target_sec:.2f}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1180	)	Cierra una estructura o llamada multilinea.
1181	elif elapsed >= MAX_LATERAL_SEC:	Evalua una condicion alternativa dentro de la misma decision.
1182	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1183	lateral_duration = elapsed	Calcula o inicializa la variable lateral_duration.
1184	finalize_attempt("lateral_timeout_obstacle_not_outside_safety_corridor")	Cierra el intento actual y guarda sus metricas.
1185	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1186	if elapsed < 1.20:	Evalua una condicion antes de ejecutar el bloque siguiente.
1187	lateral_speed = RC_LATERAL_BASE	Calcula o inicializa la variable lateral_speed.
1188	elif elapsed < 2.40:	Evalua una condicion alternativa dentro de la misma decision.
1189	lateral_speed = RC_LATERAL_STRONG	Calcula o inicializa la variable lateral_speed.
1190	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1191	lateral_speed = RC_LATERAL_MAX	Calcula o inicializa la variable lateral_speed.
1192	lr = lateral_speed if avoid_direction_sign == +1 else -lateral_speed	Calcula o inicializa la variable lr.
1193		Linea en blanco para separar bloques logicos del codigo.
1194	motion_now = time.time()	Guarda el instante actual en motion_now.
1195	dt_motion = min(max(motion_now - last_motion_update_time, 0.0), 0.20)	Calcula un valor minimo y lo asigna a dt_motion.
1196	last_motion_update_time = motion_now	Calcula o inicializa la variable last_motion_update_time.
1197	lateral_effort += abs(lr) * dt_motion	Ejecuta una instruccion necesaria para el flujo del programa.
1198		Linea en blanco para separar bloques logicos del codigo.
1199	send_smooth_rc(0, lr)	Envia un comando RC aplicando suavizado.
1200		Linea en blanco para separar bloques logicos del codigo.
1201	elif attempt_active and state == "FORWARD_AROUND_OBSTACLE":	Evalua una condicion alternativa dentro de la misma decision.
1202	bypass_elapsed = time.time() - bypass_start_time	Guarda el instante actual en bypass_elapsed.
1203	can_recheck = bypass_elapsed >= MIN_BYPASS_BEFORE_RECHECK_SEC	Calcula o inicializa la variable can_recheck.
1204	vertical_clear_during_bypass = (	Calcula o inicializa la variable vertical_clear_during_bypass.
1205	primary is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
1206	((avoidance_mode == "up" and obstacle_below_vertical_clearance(primary)) or	Ejecuta una instruccion necesaria para el flujo del programa.
1207	(avoidance_mode == "down" and obstacle_above_vertical_clearance(primary)))	Ejecuta una instruccion necesaria para el flujo del programa.
1208	)	Cierra una estructura o llamada multilinea.
1209	obstacle_reentered = (	Calcula o inicializa la variable obstacle_reentered.
1210	primary is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
1211	bbox_intersects_safety_corridor(primary) and	Ejecuta una instruccion necesaria para el flujo del programa.
1212	primary["bbox_area_norm"] > BYPASS_RECHECK_AREA_MIN and	Ejecuta una instruccion necesaria para el flujo del programa.
1213	not vertical_clear_during_bypass	Ejecuta una instruccion necesaria para el flujo del programa.
1214	)	Cierra una estructura o llamada multilinea.
1215	if can_recheck and obstacle_reentered:	Evalua una condicion antes de ejecutar el bloque siguiente.
1216	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.

Line	Code	Explanation
1217	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1218	if avoidance_mode == "up":	Evalua una condicion antes de ejecutar el bloque siguiente.
1219	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1220	set_state("AVOID_UP_UNTIL_CLEAR", "obstacle_seen_during_vertical_bypass")	Cambia el estado de la maquina de estados y lo registra.
1221	elif avoidance_mode == "down":	Evalua una condicion alternativa dentro de la misma decision.
1222	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1223	set_state("AVOID_DOWN_UNTIL_CLEAR", "obstacle_seen_during_vertical_bypass")	Cambia el estado de la maquina de estados y lo registra.
1224	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1225	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1226	set_state("AVOID_LATERAL_UNTIL_EXIT", "obstacle_reentered_safety_corridor_during_bypass")	Cambia el estado de la maquina de estados y lo registra.
1227	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1228	motion_now = time.time()	Guarda el instante actual en motion_now.
1229	dt_motion = min(max(motion_now - last_motion_update_time, 0.0), 0.20)	Calcula un valor minimo y lo asigna a dt_motion.
1230	last_motion_update_time = motion_now	Calcula o inicializa la variable last_motion_update_time.
1231		Linea en blanco para separar bloques logicos del codigo.
1232	lr_cmd = 0	Inicializa lr_cmd con un valor neutro o por defecto.
1233	if BYPASS_MODE == "diagonal" and bypass_elapsed <= BYPASS_DIAGONAL_SEC:	Evalua una condicion antes de ejecutar el bloque siguiente.
1234	lr_cmd = BYPASS_DIAGONAL_LR if avoid_direction_sign == +1 else -BYPASS_DIAGONAL_LR	Calcula o inicializa la variable lr_cmd.
1235	lateral_effort += abs(lr_cmd) * dt_motion	Ejecuta una instruccion necesaria para el flujo del programa.
1236		Linea en blanco para separar bloques logicos del codigo.
1237	forward_mps = estimate_forward_mps(RC_BYPASS_FORWARD)	Calcula o inicializa la variable forward_mps.
1238	bypass_distance_est_m += forward_mps * dt_motion	Ejecuta una instruccion necesaria para el flujo del programa.
1239		Linea en blanco para separar bloques logicos del codigo.
1240	send_smooth_rc(RC_BYPASS_FORWARD, lr_cmd)	Envia un comando RC aplicando suavizado.
1241		Linea en blanco para separar bloques logicos del codigo.
1242	if USE_DISTANCE_BASED_BYPASS:	Evalua una condicion antes de ejecutar el bloque siguiente.
1243	bypass_completed = (	Calcula o inicializa la variable bypass_completed.
1244	bypass_distance_est_m >= BYPASS_TARGET_DISTANCE_M	Ejecuta una instruccion necesaria para el flujo del programa.
1245	and bypass_elapsed >= MIN_BYPASS_FORWARD_SEC	Ejecuta una instruccion necesaria para el flujo del programa.
1246	) or bypass_elapsed >= MAX_BYPASS_FORWARD_SEC	Cierra una expresion multilinea y continua el flujo del codigo.
1247	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1248	bypass_completed = bypass_elapsed >= BYPASS_FORWARD_SEC	Calcula o inicializa la variable bypass_completed.
1249		Linea en blanco para separar bloques logicos del codigo.
1250	if bypass_completed:	Evalua una condicion antes de ejecutar el bloque siguiente.
1251	bypass_duration = bypass_elapsed	Calcula o inicializa la variable bypass_duration.
1252	recenter_clear_counter = 0	Inicializa recenter_clear_counter con un valor neutro o por defecto.
1253	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1254	set_state(	Cambia el estado de la maquina de estados y lo registra.

Line	Code	Explanation
1255	"RECENTER_AFTER_PASS",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1256	f"around_forward_completed, bypass_dist_est={bypass_distance_est_m:.2f}m, "	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1257	f"bypass_sec={bypass_duration:.2f}"	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1258	)	Cierra una estructura o llamada multilinea.
1259		Linea en blanco para separar bloques logicos del codigo.
1260	elif attempt_active and state == "RECENTER_AFTER_PASS":	Evalua una condicion alternativa dentro de la misma decision.
1261	obstacle_seen = (	Calcula o inicializa la variable obstacle_seen.
1262	primary is not None and	Ejecuta una instruccion necesaria para el flujo del programa.
1263	bbox_intersects_safety_corridor(primary) and	Ejecuta una instruccion necesaria para el flujo del programa.
1264	primary["bbox_area_norm"] > BYPASS_RECHECK_AREA_MIN	Ejecuta una instruccion necesaria para el flujo del programa.
1265	)	Cierra una estructura o llamada multilinea.
1266	if avoidance_mode == "up" and not USE_DOWN_RECENTER_AFTER_UP:	Evalua una condicion antes de ejecutar el bloque siguiente.
1267	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1268	set_state("FINAL_FORWARD", "vertical_bypass_no_down_recenter")	Cambia el estado de la maquina de estados y lo registra.
1269	elif avoidance_mode == "up" and USE_DOWN_RECENTER_AFTER_UP:	Evalua una condicion alternativa dentro de la misma decision.
1270	send_smooth_rc(0, 0, -RC_DOWN_RECENTER)	Envia un comando RC aplicando suavizado.
1271	if time.time() - state_start_time >= recenter_target_sec:	Evalua una condicion antes de ejecutar el bloque siguiente.
1272	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1273	set_state("FINAL_FORWARD", "down_recenter_completed")	Cambia el estado de la maquina de estados y lo registra.
1274	elif avoidance_mode == "down" and not USE_UP_RECENTER_AFTER_DOWN:	Evalua una condicion alternativa dentro de la misma decision.
1275	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1276	set_state("FINAL_FORWARD", "vertical_bypass_no_up_recenter")	Cambia el estado de la maquina de estados y lo registra.
1277	elif avoidance_mode == "down" and USE_UP_RECENTER_AFTER_DOWN:	Evalua una condicion alternativa dentro de la misma decision.
1278	send_smooth_rc(0, 0, RC_UP_RECENTER_AFTER_DOWN)	Envia un comando RC aplicando suavizado.
1279	if time.time() - state_start_time >= recenter_target_sec:	Evalua una condicion antes de ejecutar el bloque siguiente.
1280	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1281	set_state("FINAL_FORWARD", "up_recenter_after_down_completed")	Cambia el estado de la maquina de estados y lo registra.
1282	elif obstacle_seen:	Evalua una condicion alternativa dentro de la misma decision.
1283	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1284	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1285	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1286	set_state("AVOID_LATERAL_UNTIL_EXIT", "obstacle_seen_during_recenter")	Cambia el estado de la maquina de estados y lo registra.
1287	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1288	lr = -RC_RECENTER if avoid_direction_sign == +1 else RC_RECENTER	Calcula o inicializa la variable lr.
1289	send_smooth_rc(6, lr)	Envia un comando RC aplicando suavizado.
1290	if time.time() - state_start_time >= recenter_target_sec:	Evalua una condicion antes de ejecutar el bloque siguiente.
1291	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1292	set_state("FINAL_FORWARD", "recenter_completed")	Cambia el estado de la maquina de estados y lo registra.

Line	Code	Explanation
1293		Línea en blanco para separar bloques lógicos del código.
1294	elif attempt_active and state == "FINAL_FORWARD":	Evalúa una condición alternativa dentro de la misma decisión.
1295	obstacle_seen = (	Calcula o inicializa la variable obstacle_seen.
1296	primary is not None and	Ejecuta una instrucción necesaria para el flujo del programa.
1297	bbox_intersects_safety_corridor(primary) and	Ejecuta una instrucción necesaria para el flujo del programa.
1298	primary["bbox_area_norm"] > BYPASS_RECHECK_AREA_MIN and	Ejecuta una instrucción necesaria para el flujo del programa.
1299	not ((avoidance_mode == "up" and obstacle_below_vertical_clearance(primary)) or	Ejecuta una instrucción necesaria para el flujo del programa.
1300	(avoidance_mode == "down" and obstacle_above_vertical_clearance(primary)))	Ejecuta una instrucción necesaria para el flujo del programa.
1301	)	Cierra una estructura o llamada multilínea.
1302	if obstacle_seen:	Evalúa una condición antes de ejecutar el bloque siguiente.
1303	exit_counter = 0	Inicializa exit_counter con un valor neutro o por defecto.
1304	lost_counter = 0	Inicializa lost_counter con un valor neutro o por defecto.
1305	if avoidance_mode == "up":	Evalúa una condición antes de ejecutar el bloque siguiente.
1306	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1307	set_state("AVOID_UP_UNTIL_CLEAR", "obstacle_seen_during_final_forward")	Cambia el estado de la máquina de estados y lo registra.
1308	elif avoidance_mode == "down":	Evalúa una condición alternativa dentro de la misma decisión.
1309	up_start_time = time.time()	Guarda el instante actual en up_start_time.
1310	set_state("AVOID_DOWN_UNTIL_CLEAR", "obstacle_seen_during_final_forward")	Cambia el estado de la máquina de estados y lo registra.
1311	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1312	lateral_start_time = time.time()	Guarda el instante actual en lateral_start_time.
1313	set_state("AVOID_LATERAL_UNTIL_EXIT", "obstacle_seen_during_final_forward")	Cambia el estado de la máquina de estados y lo registra.
1314	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1315	send_smooth_rc(RC_FINAL_FORWARD, 0)	Envía un comando RC aplicando suavizado.
1316	if time.time() - state_start_time >= FINAL_FORWARD_SEC:	Evalúa una condición antes de ejecutar el bloque siguiente.
1317	stop_and_reset()	Ejecuta una instrucción necesaria para el flujo del programa.
1318	avoidance_success = True	Inicializa la bandera booleana avoidance_success.
1319	finalize_attempt("avoidance_completed_v6_distance_bypass_effort_recenter")	Cierra el intento actual y guarda sus métricas.
1320	else:	Ejecuta este bloque cuando las condiciones anteriores no se cumplen.
1321	if flying:	Evalúa una condición antes de ejecutar el bloque siguiente.
1322	stop_and_reset()	Ejecuta una instrucción necesaria para el flujo del programa.
1323		Línea en blanco para separar bloques lógicos del código.
1324		Línea en blanco para separar bloques lógicos del código.
1325	center_min_px = int(CENTER_X_MIN * W)	Convierte el resultado a entero y lo guarda en center_min_px.
1326	center_max_px = int(CENTER_X_MAX * W)	Convierte el resultado a entero y lo guarda en center_max_px.
1327	safety_min_px = int(SAFETY_X_MIN * W)	Convierte el resultado a entero y lo guarda en safety_min_px.
1328	safety_max_px = int(SAFETY_X_MAX * W)	Convierte el resultado a entero y lo guarda en safety_max_px.
1329	cv2.rectangle(frame, (safety_min_px, 0), (safety_max_px, H), (255, 255, 0), 2)	Dibuja un rectángulo sobre el frame.
1330	cv2.rectangle(frame, (center_min_px, 0), (center_max_px, H), (255, 180, 0), 1)	Dibuja un rectángulo sobre el frame.

Line	Code	Explanation
1331	if ENABLE_UPWARD_AVOIDANCE:	Evalua una condicion antes de ejecutar el bloque siguiente.
1332	min_top_y_px = int(UP_MIN_TOP_CLEARANCE * H)	Convierte el resultado a entero y lo guarda en min_top_y_px.
1333	head_zone_y_px = int(UP_HEAD_Y1_MAX * H)	Convierte el resultado a entero y lo guarda en head_zone_y_px.
1334	head_center_y_px = int(UP_HEAD_CENTER_Y_MAX * H)	Convierte el resultado a entero y lo guarda en head_center_y_px.
1335	clear_top_y_px = int(UP_CLEAR_TOP_Y_NORM * H)	Convierte el resultado a entero y lo guarda en clear_top_y_px.
1336	cv2.line(frame, (0, min_top_y_px), (W, min_top_y_px), (180, 180, 255), 1)	Dibuja una linea sobre el frame.
1337	cv2.line(frame, (0, head_zone_y_px), (W, head_zone_y_px), (0, 180, 255), 2)	Dibuja una linea sobre el frame.
1338	cv2.line(frame, (0, head_center_y_px), (W, head_center_y_px), (0, 140, 255), 1)	Dibuja una linea sobre el frame.
1339	cv2.line(frame, (0, clear_top_y_px), (W, clear_top_y_px), (0, 255, 255), 1)	Dibuja una linea sobre el frame.
1340	cv2.putText(frame, "UP head y1 zone", (12, max(18, head_zone_y_px - 5)), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 180, 255), 1)	Escribe texto informativo sobre el frame.
1341	cv2.putText(frame, "UP head cy limit", (12, max(18, head_center_y_px - 5)), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 140, 255), 1)	Escribe texto informativo sobre el frame.
1342	cv2.putText(frame, "UP clear line", (12, max(18, clear_top_y_px - 5)), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 255, 255), 1)	Escribe texto informativo sobre el frame.
1343	if ENABLE_DOWNWARD_AVOIDANCE:	Evalua una condicion antes de ejecutar el bloque siguiente.
1344	min_bottom_y_px = int((1.0 - DOWN_MIN_BOTTOM_CLEARANCE) * H)	Convierte el resultado a entero y lo guarda en min_bottom_y_px.
1345	clear_bottom_y_px = int(DOWN_CLEAR_BOTTOM_Y_NORM * H)	Convierte el resultado a entero y lo guarda en clear_bottom_y_px.
1346	cv2.line(frame, (0, min_bottom_y_px), (W, min_bottom_y_px), (255, 180, 180), 1)	Dibuja una linea sobre el frame.
1347	cv2.line(frame, (0, clear_bottom_y_px), (W, clear_bottom_y_px), (0, 180, 255), 1)	Dibuja una linea sobre el frame.
1348	cv2.putText(frame, "DOWN min bottom clearance", (12, min(H - 8, min_bottom_y_px + 16)), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (255, 180, 180), 1)	Escribe texto informativo sobre el frame.
1349	cv2.putText(frame, "DOWN clear line", (12, min(H - 8, clear_bottom_y_px + 16)), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 180, 255), 1)	Escribe texto informativo sobre el frame.
1350	draw_overlay(	Ejecuta una instruccion necesaria para el flujo del programa.
1351	frame,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1352	flying,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1353	state,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1354	scenario_name,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1355	attempt_active,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1356	primary,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1357	distance_est,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1358	ttc_proxy,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1359	danger,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1360	warning,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1361	exit_counter,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1362	lost_counter,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1363	lateral_effort,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1364	recenter_target_sec,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1365	bypass_distance_est_m,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.

Line	Code	Explanation
1366	avoidance_mode,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1367	vertical_effort,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1368	up_duration,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1369	)	Cierra una estructura o llamada multilinea.
1370	if awaiting_operator_label:	Evalua una condicion antes de ejecutar el bloque siguiente.
1371	cv2.putText(frame, "LABEL REQUIRED: Y=success F=failure U=unclear", (12, H - 45), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)	Escribe texto informativo sobre el frame.
1372	cv2.putText(frame, f"Labelled trials: {labelled_trials} Operator successes: {labelled_successes}", (12, H - 18), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2)	Escribe texto informativo sobre el frame.
1373		Linea en blanco para separar bloques logicos del codigo.
1374	if SAVE_PERIODIC_FRAMES and (time.time() - last_frame_save >= PERIODIC_FRAME_INTERVAL_SEC):	Evalua una condicion antes de ejecutar el bloque siguiente.
1375	cv2.imwrite(str(run_dir / "frames_raw" / f"frame_{frame_idx:06d}_raw.jpg"), raw_frame)	Ejecuta una instruccion necesaria para el flujo del programa.
1376	cv2.imwrite(str(run_dir / "frames_annotated" / f"frame_{frame_idx:06d}_ann.jpg"), frame)	Ejecuta una instruccion necesaria para el flujo del programa.
1377	last_frame_save = time.time()	Guarda el instante actual en last_frame_save.
1378		Linea en blanco para separar bloques logicos del codigo.
1379	if time.time() - last_telemetry_log >= TELEMETRY_LOG_INTERVAL_SEC:	Evalua una condicion antes de ejecutar el bloque siguiente.
1380	telemetry_writer.writerow([	Escribe una fila en un archivo CSV de registro.
1381	timestamp, frame_idx, attempt_id if attempt_active else -1, scenario_name, PROFILE_NAME,	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1382	safe_call(tello.get_battery), safe_call(tello.get_height), safe_call(tello.get_distance_tof),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1383	safe_call(tello.get_yaw), safe_call(tello.get_pitch), safe_call(tello.get_roll),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1384	safe_call(tello.get_speed_x), safe_call(tello.get_speed_y), safe_call(tello.get_speed_z),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1385	state, int(flying), int(attempt_active),	Elemento dentro de una lista, tupla, diccionario o llamada multilinea.
1386	f"{lateral_effort:.3f}", f"{bypass_distance_est_m:.3f}", f"{recenter_target_sec:.3f}",	Cadena usada como texto, mensaje, etiqueta o elemento de una lista.
1387	avoidance_mode, f"{vertical_effort:.3f}", f"{up_duration:.3f}"	Ejecuta una instruccion necesaria para el flujo del programa.
1388	])	Cierra una expresion multilinea y continua el flujo del codigo.
1389	telemetry_f.flush()	Fuerza el guardado inmediato de los datos pendientes.
1390	last_telemetry_log = time.time()	Guarda el instante actual en last_telemetry_log.
1391		Linea en blanco para separar bloques logicos del codigo.
1392	if ann_writer is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1393	ann_writer.write(frame)	Ejecuta una instruccion necesaria para el flujo del programa.
1394	if raw_writer is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1395	raw_writer.write(raw_frame)	Ejecuta una instruccion necesaria para el flujo del programa.
1396		Linea en blanco para separar bloques logicos del codigo.
1397	cv2.imshow(WINDOW_NAME, cv2.resize(frame, (960, 540)))	Muestra el frame anotado en la ventana de OpenCV.
1398	# Manual interface	Comentario corto que identifica la seccion del codigo.
1399	key = cv2.waitKeyEx(1)	Calcula o inicializa la variable key.
1400		Linea en blanco para separar bloques logicos del codigo.
1401		Linea en blanco para separar bloques logicos del codigo.
1402	if key in (27, ord('q'), ord('Q')):	Evalua una condicion antes de ejecutar el bloque siguiente.

Line	Code	Explanation
1403	if attempt_active:	Evalua una condicion antes de ejecutar el bloque siguiente.
1404	finalize_attempt("quit_during_attempt")	Cierra el intento actual y guarda sus metricas.
1405	break	Sale del bucle actual.
1406	elif key in (ord('t'), ord('T')):	Evalua una condicion alternativa dentro de la misma decision.
1407	if not flying:	Evalua una condicion antes de ejecutar el bloque siguiente.
1408	try:	Inicia un bloque protegido frente a errores de ejecucion.
1409	tello.takeoff()	Ordena el despegue del dron.
1410	time.sleep(TAKEOFF_STABILIZE_SEC)	Ejecuta una instruccion necesaria para el flujo del programa.
1411	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1412	flying = True	Inicializa la bandera booleana flying.
1413	state = "HOVER"	Calcula o inicializa la variable state.
1414	log_event(frame_idx, "takeoff_ok", f"battery={safe_call(tello.get_battery)}")	Lee el nivel de bateria del dron.
1415	except Exception as e:	Gestiona una excepcion producida dentro del bloque try.
1416	log_event(frame_idx, "takeoff_error", str(e))	Registra un evento relevante del experimento.
1417	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1418	elif key in (ord('l'), ord('L')):	Evalua una condicion alternativa dentro de la misma decision.
1419	if attempt_active:	Evalua una condicion antes de ejecutar el bloque siguiente.
1420	finalize_attempt("manual_land_during_attempt")	Cierra el intento actual y guarda sus metricas.
1421	if flying:	Evalua una condicion antes de ejecutar el bloque siguiente.
1422	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1423	try:	Inicia un bloque protegido frente a errores de ejecucion.
1424	tello.land()	Ordena el aterrizaje del dron.
1425	except Exception as e:	Gestiona una excepcion producida dentro del bloque try.
1426	log_event(frame_idx, "land_error", str(e))	Registra un evento relevante del experimento.
1427	flying = False	Inicializa la bandera booleana flying.
1428	state = "IDLE"	Calcula o inicializa la variable state.
1429	elif key == ord('1'):	Evalua una condicion alternativa dentro de la misma decision.
1430	scenario_name = "centered_person"	Calcula o inicializa la variable scenario_name.
1431	expected_obstacle_class = "person"	Calcula o inicializa la variable expected_obstacle_class.
1432	elif key == ord('2'):	Evalua una condicion alternativa dentro de la misma decision.
1433	scenario_name = "left_person"	Calcula o inicializa la variable scenario_name.
1434	expected_obstacle_class = "person"	Calcula o inicializa la variable expected_obstacle_class.
1435	elif key == ord('3'):	Evalua una condicion alternativa dentro de la misma decision.
1436	scenario_name = "right_person"	Calcula o inicializa la variable scenario_name.
1437	expected_obstacle_class = "person"	Calcula o inicializa la variable expected_obstacle_class.
1438	elif key == ord('4'):	Evalua una condicion alternativa dentro de la misma decision.
1439	scenario_name = "centered_car"	Calcula o inicializa la variable scenario_name.
1440	expected_obstacle_class = "car"	Calcula o inicializa la variable expected_obstacle_class.

Line	Code	Explanation
1441	<code>elif key == ord('5'):</code>	Evalua una condicion alternativa dentro de la misma decision.
1442	<code>scenario_name = "centered_person_upward_escape"</code>	Calcula o inicializa la variable <code>scenario_name</code> .
1443	<code>expected_obstacle_class = "person"</code>	Calcula o inicializa la variable <code>expected_obstacle_class</code> .
1444	<code>elif key == ord('6'):</code>	Evalua una condicion alternativa dentro de la misma decision.
1445	<code>scenario_name = "centered_person_downward_escape"</code>	Calcula o inicializa la variable <code>scenario_name</code> .
1446	<code>expected_obstacle_class = "person"</code>	Calcula o inicializa la variable <code>expected_obstacle_class</code> .
1447	<code>elif key in (ord('y'), ord('Y')):</code>	Evalua una condicion alternativa dentro de la misma decision.
1448	<code>label_last_attempt("success", True)</code>	Guarda la etiqueta manual del intento anterior.
1449	<code>elif key in (ord('f'), ord('F')):</code>	Evalua una condicion alternativa dentro de la misma decision.
1450	<code>label_last_attempt("failure", False)</code>	Guarda la etiqueta manual del intento anterior.
1451	<code>elif key in (ord('u'), ord('U')):</code>	Evalua una condicion alternativa dentro de la misma decision.
1452	<code>label_last_attempt("unclear", None, "invalid_or_unclear_trial")</code>	Guarda la etiqueta manual del intento anterior.
1453	<code>elif key == 13:</code>	Evalua una condicion alternativa dentro de la misma decision.
1454	<code>start_attempt()</code>	Inicia un nuevo intento experimental.
1455	<code>elif key in (ord('x'), ord('X')):</code>	Evalua una condicion alternativa dentro de la misma decision.
1456	<code>if attempt_active:</code>	Evalua una condicion antes de ejecutar el bloque siguiente.
1457	<code>finalize_attempt("manual_abort")</code>	Cierra el intento actual y guarda sus metricas.
1458	<code>elif key == ARROW_UP and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1459	<code>tello.send_rc_control(0, RC_MANUAL_SPEED, 0, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1460	<code>elif key == ARROW_DOWN and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1461	<code>tello.send_rc_control(0, -RC_MANUAL_SPEED, 0, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1462	<code>elif key == ARROW_LEFT and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1463	<code>tello.send_rc_control(-RC_MANUAL_SPEED, 0, 0, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1464	<code>elif key == ARROW_RIGHT and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1465	<code>tello.send_rc_control(RC_MANUAL_SPEED, 0, 0, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1466	<code>elif key in (ord('w'), ord('W')) and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1467	<code>tello.send_rc_control(0, 0, RC_UPDOWN_SPEED, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1468	<code>elif key in (ord('s'), ord('S')) and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1469	<code>tello.send_rc_control(0, 0, -RC_UPDOWN_SPEED, 0)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1470	<code>elif key in (ord('a'), ord('A')) and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1471	<code>tello.send_rc_control(0, 0, 0, -RC_YAW_SPEED)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1472	<code>elif key in (ord('d'), ord('D')) and flying and not attempt_active:</code>	Evalua una condicion alternativa dentro de la misma decision.
1473	<code>tello.send_rc_control(0, 0, 0, RC_YAW_SPEED)</code>	Envia comandos RC al dron en formato izquierda-derecha, avance-retroceso, vertical y yaw.
1474	<code>elif not attempt_active and key == -1:</code>	Evalua una condicion alternativa dentro de la misma decision.
1475	<code>if flying:</code>	Evalua una condicion antes de ejecutar el bloque siguiente.
1476	<code>stop_and_reset()</code>	Ejecuta una instruccion necesaria para el flujo del programa.
1477		Linea en blanco para separar bloques logicos del codigo.
1478	<code>frame_idx += 1</code>	Ejecuta una instruccion necesaria para el flujo del programa.

Line	Code	Explanation
1479		Linea en blanco para separar bloques logicos del codigo.
1480	except Exception as e:	Gestiona una excepcion producida dentro del bloque try.
1481	traceback.print_exc()	Ejecuta una instruccion necesaria para el flujo del programa.
1482	try:	Inicia un bloque protegido frente a errores de ejecucion.
1483	log_event(frame_idx, "exception", str(e))	Registra un evento relevante del experimento.
1484	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1485	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1486	finally:	Ejecuta tareas finales aunque haya ocurrido un error.
1487	# Cleanup	Comentario corto que identifica la seccion del codigo.
1488	try:	Inicia un bloque protegido frente a errores de ejecucion.
1489	if attempt_active:	Evalua una condicion antes de ejecutar el bloque siguiente.
1490	finalize_attempt("cleanup_finalize")	Cierra el intento actual y guarda sus metricas.
1491	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1492	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1493	try:	Inicia un bloque protegido frente a errores de ejecucion.
1494	stop_and_reset()	Ejecuta una instruccion necesaria para el flujo del programa.
1495	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1496	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1497	try:	Inicia un bloque protegido frente a errores de ejecucion.
1498	if flying:	Evalua una condicion antes de ejecutar el bloque siguiente.
1499	tello.land()	Ordena el aterrizaje del dron.
1500	except Exception as e:	Gestiona una excepcion producida dentro del bloque try.
1501	try:	Inicia un bloque protegido frente a errores de ejecucion.
1502	log_event(frame_idx, "cleanup_land_error", str(e))	Registra un evento relevante del experimento.
1503	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1504	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1505	try:	Inicia un bloque protegido frente a errores de ejecucion.
1506	tello.streamoff()	Desactiva el flujo de video del dron.
1507	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1508	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1509	try:	Inicia un bloque protegido frente a errores de ejecucion.
1510	tello.end()	Cierra la conexion con el dron.
1511	except Exception:	Gestiona una excepcion producida dentro del bloque try.
1512	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1513	for fobj in (events_f, attempts_f, detections_f, telemetry_f, labels_f):	Inicia un bucle sobre una secuencia de elementos.
1514	try:	Inicia un bloque protegido frente a errores de ejecucion.
1515	fobj.close()	Cierra el archivo abierto.
1516	except Exception:	Gestiona una excepcion producida dentro del bloque try.

Line	Code	Explanation
1517	pass	Ejecuta una instruccion necesaria para el flujo del programa.
1518	if ann_writer is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1519	ann_writer.release()	Libera el escritor de video.
1520	if raw_writer is not None:	Evalua una condicion antes de ejecutar el bloque siguiente.
1521	raw_writer.release()	Libera el escritor de video.
1522	cv2.destroyAllWindows()	Cierra todas las ventanas de OpenCV.
1523		Linea en blanco para separar bloques logicos del codigo.
1524		Linea en blanco para separar bloques logicos del codigo.
1525	if __name__ == "__main__":	Comprueba que el archivo se esta ejecutando directamente.
1526	main()	Ejecuta una instruccion necesaria para el flujo del programa.